



**HARDWARE ACCELERATED EMG BASED TRANSFORMER
MODEL FOR HAND PROSTHETICS**

Abdullah Amir

Muhammad Quddusi Kashaf

Saif Nazir

**HABIB UNIVERSITY
KARACHI, PAKISTAN**

2026

HARDWARE ACCELERATED EMG BASED TRANSFORMER MODEL FOR HAND PROSTHETICS

The Capstone Design Project
presented to the academic faculty

by

Abdullah Amir

Muhammad Quddusi Kashaf

Saif Nazir

in fulfillment of the requirements for
BS Computer Engineering
in the
School of Science and Engineering

Habib University

May 2026



This work is licensed under a Creative Commons Attribution 3.0 License.

HARDWARE ACCELERATED EMG BASED TRANSFORMER MODEL FOR HAND PROSTHETICS

This capstone design project was advised by:

Dr. Hamood ur Rehman
Department of Electrical and
Computer Engineering
Habib University

Approved by the Department of Electrical and Computer Engineering on

Once you have tasted flight, you will forever walk the earth with your eyes turned skyward, for there you have been, and there you will always long to return.

John H. Secondari

Dedicated to our families, whose unwavering support provided the foundation for our growth, and to the resilient spirit of those we aim to serve through technology.

ACKNOWLEDGEMENTS

We are deeply grateful to the faculty at **Habib University** for challenging us, and for the Liberal Core in our curriculum, which cultivated the passion and spirit of service in our work. Not to forget, the Research Assistants at the **Department of Electrical and Computer Engineering** at **Habib University** who issued us the Zedboard.

A special note of appreciation goes to our colleagues and peers for the late-night brainstorming sessions and for providing a sounding board during our most challenging hardware-software integration sprints.

Finally, many thanks to our families and friends whose patience and encouragement were the silent engines behind the countless hours spent debugging AXI interfaces, training Transformer models, and perfecting our visualization.

TABLE OF CONTENTS

List of Tables	xi
List of Figures	xii
Chapter 1: Introduction	1
1.1 Background	1
1.2 Relevance of the Problem	1
1.3 Preliminaries	2
1.3.1 Human Motor Control and Muscle Activation	2
1.3.2 Electromyography (EMG): What It Is and What It Measures	2
1.3.3 Challenges in Interpreting sEMG Signals	3
1.3.4 Role of Machine Learning in EMG Interpretation	4
1.3.5 Real-Time Requirements and Embedded Processing Considerations	4
1.4 Literature Review	5
1.4.1 Commercial Prosthetics & Myoelectric Hands	5
1.4.2 Conventional and Classical ML Approaches	6
1.4.3 Deep Learning in Myoelectric Control (CNNs & RNNs)	6
1.4.4 The Rise of Transformers for Sequential Data	7
1.4.5 The Deployment Challenge: AI at the Edge & FPGAs	7
1.5 Stakeholder Analysis	7

1.6	Sustainable Development Goals	8
1.7	Detailed Problem Description	8
1.7.1	Desired Features and Objectives	8
1.8	Functional Requirements	9
1.9	Constraints & Limitations	9
Chapter 2:	Design Concepts	11
2.1	Design Alternatives	11
2.1.1	Morphological Analysis	11
2.1.2	Discussion of Alternatives	12
2.2	Metrics for Objectives: Success Criteria	13
2.3	Critical Design Issues & Mitigation	13
Chapter 3:	Design Details	14
3.1	Project Focus	14
3.2	Technical Challenge	14
3.3	Technical Requirements	15
3.4	Societal Considerations	15
3.4.1	Healthcare Accessibility	15
3.4.2	Data Privacy (Ethics)	16
3.5	Environment and Sustainability	16
3.5.1	Power Efficiency	16
3.5.2	Hardware Longevity	16
3.5.3	Reduced Carbon Footprint via Edge Computing	16
3.5.4	Algorithmic “Green AI”	16

3.6	Intellectual Contribution	17
Chapter 4: Design Details		19
4.1	High Level System Design	19
4.2	Software Subsystem	20
4.2.1	Data Analysis	20
4.2.2	sEMG Signal Conditioning and Preprocessing	24
4.2.3	Model Architecture and Training	25
4.3	Hardware Design and Acceleration	29
4.3.1	Design using High-Level Synthesis (HLS)	29
4.3.2	Vivado Block Design and AXI Interfacing	31
4.4	Communication and Integration Layer	32
4.4.1	UART Communication	32
4.4.2	Unity XR Hand Integration	33
4.4.3	GUI Architecture and Visualization Pipeline	33
4.4.4	Post-Processing: Joint Angle Smoothing	35
Chapter 5: Prototyping and Testing		38
5.1	Prototype Overview	38
5.2	Testing Methodology	38
5.2.1	Model Evaluation and Performance Metrics	38
5.2.2	FPGA Transformer Inference Verification	39
5.2.3	Visualization Pipeline Verification	40
5.3	Key Performance Indicators (KPIs)	40
5.4	Results	41

5.4.1	Inference Accuracy	41
5.4.2	FPGA Resource Utilization and Power	42
5.4.3	Post-processing Ablation	42
5.4.4	Visualization Performance	42
Appendix A: Material and Component Details		44
A.1	Datasheet References	44
A.2	System Requirements	44
A.2.1	Hardware Requirements	44
A.2.2	Software Requirements	45
Appendix B: Project Schedule		46
Appendix B: Project Schedule		46
Appendix C: Hardware Setup and Operating Procedure		49
C.1	ZedBoard Physical Setup	49
C.2	Bitstream Generation and Programming	50
C.2.1	Generating the Bitstream in Vivado	50
C.2.2	Programming the ZedBoard	50
C.3	Running the Full Pipeline	50
C.4	Troubleshooting	51
Appendix D: Block Design and Schematics		52
Appendix D: Block Design and Schematics		52
Appendix E: Selected Source Code		55

Appendix F: Selected Source Code	55
Bibliography	78
Vita	79

LIST OF TABLES

2.1	Expanded Morphological Chart for sEMG Gesture Recognition System	11
4.1	The 10 target joint angles estimated by the model and their anatomical descriptions.	23
4.2	Model hyperparameters and training configuration.	29
4.3	Standard Kalman filter hyperparameters.	36
4.4	Adaptive Kalman filter hyperparameters.	37
5.1	KPIs and design targets for prototype evaluation.	40
5.2	Per-joint RMSE — software FP32 vs. FPGA INT8.	41
5.3	FPGA resource utilization (Vivado implementation report).	42
A.1	Bill of Materials (BOM) for prototype.	44
B.1	Work package summary.	48
C.1	Common issues and resolutions.	51
D.1	AXI master address assignments for the RoFormer_Top_0 IP (Network 0). Both <code>m_axi_gmem0</code> and <code>m_axi_gmem1</code> share the same address map.	53

LIST OF FIGURES

4.1	High-level system architecture linking EMG input, FPGA inference, and Unity visualization.	19
4.2	Per-subject distribution of representative joint-angle values across the DB2 dataset.	20
4.3	10 joint angles of interest (highlighted in red)	22
4.4	Anatomical Structure of Hand	23
4.5	RoFormer model architecture: 1D-CNN input head, RoPE positional encoding, two transformer encoder blocks with residual connections, and a linear output head regressing 10 joint angles. The velocity-guided loss supervises both position and velocity during training.	26
5.1	Predicted vs. ground-truth joint angles for a representative 200 ms test window (FPGA INT8 output).	41
B.1	Project Gantt chart covering Capstone 1 (Aug 2025 – Dec 2025) and Capstone 2 (Jan 2026 – May 2026). Red diamonds mark key milestones; blue bars indicate planned work packages.	47
C.1	ZedBoard jumper configuration for SD card boot mode.	49
D.1	Vivado IP Integrator block design. The RoFormer_Top_0 HLS accelerator (left) connects to the ZYNQ7 Processing System (right) via a three-slave AXI Interconnect for high-bandwidth DDR access and an AXI SmartConnect for low-latency control register access. The AXI GPIO peripheral drives the status LEDs on the ZedBoard.	53
D.2	Vivado Address Editor – Network 0. Both AXI master ports of the RoFormer HLS IP are mapped to three contiguous 128 MB DDR segments through the PS high-performance slave ports, giving the accelerator direct burst access to the full inference data region defined in <code>main.c</code>	54

EXECUTIVE SUMMARY

Thousands of individuals in Pakistan face limb amputations annually, yet access to functional prosthetic hands is severely limited. Current solutions present a stark choice: either rudimentary, body-powered mechanical devices with minimal functionality or advanced, imported myoelectric prosthetics whose multi-thousand-dollar cost is prohibitive for most of the population.

This project aims to design, develop, and implement an affordable, high-performance, non-invasive prosthetic hand control system based on Electromyographic (EMG) signals.

The core technical challenge is achieving real-time, high-accuracy gesture recognition on a power-efficient, portable (edge) device. Advanced AI models like Transformers are exceptionally good at understanding complex time-series data like EMG signals, but they are computationally intensive.

Our solution directly confronts this challenge by implementing a lightweight, quantized Transformer model on a Field-Programmable Gate Array (FPGA). The FPGA's inherent parallelism and power efficiency make it the ideal platform to accelerate the Transformer's computations for real-time, on-device inference.

KEYWORDS: Hardware Acceleration, Transformer Model, Prosthetic Hands

CHAPTER 1

INTRODUCTION

1.1 Background

Every year, thousands of individuals in Pakistan suffer limb amputations due to traumatic accidents, electrical burns, and medical complications such as diabetes [1]. For these individuals, regaining functionality is a significant challenge. While advanced prosthetic technology exists, a massive “accessibility gap” divides the available solutions.

On one end of the spectrum are rudimentary, body-powered devices. These are often limited to simple hook or pincer grips, which are fatiguing to use, lack natural dexterity, and offer limited functionality. On the other are advanced, imported myoelectric prosthetics. These devices use muscle signals (Electromyography, or EMG) to control multi-articulated fingers, offering intuitive and natural movement. However, with costs ranging from \$10,000 to over \$100,000, they remain financially unattainable for the vast majority of the population in developing countries, creating significant healthcare and social inequality [2].

Furthermore, while model-based hand movement prediction exists, the most powerful models, such as Transformers, are computationally heavy, demanding significant memory and processing power [3]. Consequently, their application in hardware-accelerated, real-time edge devices remains vastly unexplored, presenting a critical research gap and a significant opportunity to leverage their power for new applications.

1.2 Relevance of the Problem

Pakistan, is the fifth most populous country in the world and has a high rate of limb loss due to industrial accidents, traffic incidents, and conflict injuries. Unfortunately, the country also suffers from the highest pervalence of diabetes in the world, which

is known for causing neuropathy (nerve damage due to high levels of blood sugar), that can lead to limb loss. These circumstances result in a large number of people struggling with limb amputations. The most common solutions to these amputations such as prosthetics or bionic limbs are largely unavailable due to a lack of local industry and R&D for biomedical engineering in the country. Imported alternatives for these are very expensive and remain out of grasp of the average Pakistani.

With such circumstances paired with a struggling economy, the inability to afford a functional prosthetic is not merely an inconvenience; it is a critical socio-economic barrier that is a primary factor in device abandonment [4].

1.3 Preliminaries

To understand the problem better, we must first understand the core concepts that drive limb movement:

1.3.1 Human Motor Control and Muscle Activation

Human movement is generated through a coordinated cascade of neural processes. It travels as motor commands from the brain through the spinal cord and peripheral nerves toward the relevant muscles. These neural impulses cause motor units within each muscle to depolarize and produce contractions of varying intensity, known as Motor Unit Action Potentials (MUAPs). The pattern and strength of these contractions correspond directly to the intended movement, such as closing a fist, extending a finger, or rotating the wrist. Although the mechanical action occurs at the muscles, the underlying electrical activation provides a measurable window into a person's motor intent.

1.3.2 Electromyography (EMG): What It Is and What It Measures

Electromyography (EMG) refers to the measurement of this electrical activity generated by muscle fibers during contraction. The most commonly used form in non-invasive systems is *surface EMG (sEMG)*, where electrodes are placed on the skin to detect the cumulative voltage fluctuations of multiple MUAPs [5].

Following are some of the main properties of EMG signals:

- **Amplitude:** Typically 0–10 mV peak-to-peak, which reflects muscle activation intensity[6].
- **Frequency:** Most energy is concentrated between 0 Hz–500 Hz[6].
- **Spatial Selectivity:** sEMG captures the combined activity of multiple muscles and can overlap when muscles lie close together[7].
- **Temporal Dynamics:** EMG responds rapidly to changes in intention (a few tens of milliseconds)[8].

1.3.3 Challenges in Interpreting sEMG Signals

Decoding movements from raw surface EMG signals is inherently challenging:

- **Signal Noise and Artifacts:** sEMG is highly susceptible to power-line interference, motion artifacts, electrode micro-movements, and cross-talk between adjacent muscles [9].
- **Non-linearity of Muscle Activation:** The relationship between EMG amplitude and force, joint angle, or movement is non-linear and varies across individuals.
- **Inter- and Intra-Subject Variability:** EMG patterns differ significantly between people and even within the same person across time due to fatigue, skin impedance, sweat, or electrode placement differences.
- **Temporal Complexity:** Hand movements involve rapid, coordinated movements of multiple muscles maybe me much difficult to decode.
- **Redundancy and Coupling:** Multiple muscles may activate for the same movement, and a single muscle may participate in several different motions.

1.3.4 Role of Machine Learning in EMG Interpretation

Machine Learning (ML) is a branch of artificial intelligence that enables computers to identify patterns and relationships in data without being explicitly programmed for specific rules. ML algorithms learn from examples to make predictions or decisions, making them particularly suitable for complex, high-dimensional, and noisy data.

In context of EMG-based hand motion tracking, ML can be useful because:

- **Capturing Complex Patterns:** EMG signals are non-linear and vary over time. ML models can learn these intricate relationships between the signal and intended movements.
- **Handling Noise and Variability:** Muscle signals are affected by electrode placement, skin properties, and external noise. ML methods can generalize across these variations better than simple rule-based approaches.
- **Continuous Prediction:** Instead of limiting output to predefined gestures, ML can predict continuous hand motion, allowing more natural and precise control.
- **Adaptability:** ML models can be retrained or fine-tuned to accommodate differences between individuals or changing conditions over time.

With the advent of Deep learning models, these useful representations can be learnt directly from raw or minimally processed EMG. Convolutional models are able to extract spatial and frequency features, while recurrent models and transformers can learn temporal dependencies across sequences. This allows for continuous estimation of joint angles, grip forces, and finger positions rather than limiting control to coarse gesture categories [10].

1.3.5 Real-Time Requirements and Embedded Processing Considerations

Systems using EMG to control a prosthetic hand, virtual limb, or rehabilitation interface all need to operate with low latency to feel intuitive. Delays greater than approximately 100–150 ms can disrupt a user’s sense of agency, making the system feel unnatural.

To be **portable, affordable, and locally manufacturable**, the computational workload must be executed on embedded hardware with limited power and memory.

1.4 Literature Review

1.4.1 Commercial Prosthetics & Myoelectric Hands

Over the past few decades, multi-articulating prosthetic hands using myoelectric control have emerged as one of the most advanced options for upper-limb amputees. These devices drive motors using electromyographic (EMG) signals to actuate multiple independent fingers or joints, enabling a range of prehensile grips and movements [11].

- The i-Limb Hand (and its variants) is a widely used myoelectric prosthesis offering multiple grip patterns, powered digits, and a rotating thumb [12].
- The BeBionic Hand (produced by Ottobock) is another major myoelectric prosthetic with individually powered digits designed to give users a higher level of dexterity than simple hooks or passive prosthetics [13].
- There have also been multiple 3D printed variants for functional myoelectric-hand designs developed which provide an inexpensive solution to the problem but at the cost of limited functionality [14].

These devices represent the state-of-the-art in prosthetics for restoring hand function, offering far greater flexibility and functionality than traditional body-powered or cosmetic prostheses. But despite their promise, they still face several important limitations such as:

- **Affordability Barrier:** High-quality myoelectric hands are typically expensive. Prosthetics like i-Limb or BeBionic often cost more than what many patients in low-income settings can afford [13].
- **Calibration Requirements:** Users often need prior training and adjustment to use prosthetic hands effectively. The pattern recognition or control interfaces

may require repeated calibrations, and some control modes remain unintuitive for daily tasks [13].

- **Functional Limitations in Real-World Use:** Purely mechanical or EMG-controlled prosthetics often struggle to perform reliable fine motor tasks or adaptive grips under variable conditions (e.g. variable load, complex hand postures). They are only able to provide basic support or stabilizing function for daily living tasks [13].

1.4.2 Conventional and Classical ML Approaches

The most basic functional prosthetics are **body-powered**, which have not changed significantly in decades. They are reliable and low-cost but are non-intuitive and mechanically limited [2]. The first generation of **myoelectric** hands relied on classical machine learning (ML). In these systems, engineers would manually extract statistical features from the raw EMG signal (e.g., Mean Absolute Value, Root Mean Square). These features would then be fed into a simple classifier, such as a Support Vector Machine (SVM) or k-Nearest Neighbors (kNN), to predict a gesture [15]. This approach is brittle; its performance depends heavily on hand-crafted features, which do not generalize well.

1.4.3 Deep Learning in Myoelectric Control (CNNs & RNNs)

The advent of deep learning led to significant improvements. Models like **Convolutional Neural Networks (CNNs)** and **Recurrent Neural Networks (RNNs)** could learn relevant features automatically from the raw EMG data [16]. CNNs are adept at finding spatial patterns, while RNNs are designed for temporal sequences. However, CNNs often struggle with long-range temporal dependencies, and RNNs are computationally slow due to their sequential nature.

1.4.4 The Rise of Transformers for Sequential Data

Given the limitations of CNNs and RNNs, recent research has begun to apply the **Transformer** architecture to myoelectric control. Studies by Wei et al. [17] and Park and Lee [18] have demonstrated that Transformer-based models can outperform state-of-the-art CNNs and RNNs by better understanding the complex temporal dynamics of muscle activation.

1.4.5 The Deployment Challenge: AI at the Edge & FPGAs

Despite their accuracy, Transformers are computationally massive [19, 3]. They traditionally run on GPUs, which are unsuitable for a portable, battery-operated prosthetic that demands low latency, low power, and on-device processing. This is where **Field-Programmable Gate Arrays (FPGAs)** provide a way forward. They have a flexible hardware architecture that can be reconfigured to create a custom-built digital circuit, enabling massive parallelism and optimization at a fraction of the power cost of a GPU [20]. While research exists on accelerating NLP-based Transformers on FPGAs [21], a significant gap remains in developing accelerators for time-series models, specifically in real-time biomedical applications.

1.5 Stakeholder Analysis

Based on preliminary research, we have identified four primary stakeholder groups around which we have shaped the objectives of this project:

- **Amputees:** They require intuitive, low-cost, reliable control with minimal time for calibration.
- **Physiotherapists/Hospitals:** An accessible rehabilitation tools and EMG-based feedback systems may prove efficient in therapy.
- **Researchers:** A need for reproducible pipeline involving sEMG, efficient hardware and a light-weight deep learning model.

- **Manufacturers:** A need for scalable embedded designs suitable for local production.

1.6 Sustainable Development Goals

This project directly addresses several of the United Nations Sustainable Development Goals:

- **SDG 3: Good Health and Well-Being:** By creating accessible and affordable assistive technology, this project supports rehabilitation and aims to improve the quality of life for individuals with physical disabilities.
- **SDG 8: Decent Work and Economic Growth:** A functional prosthetic is an enabling technology, allowing individuals to rejoin the workforce, fostering inclusive employment, and promoting economic self-sufficiency.
- **SDG 9: Industry, Innovation, and Infrastructure:** This project pushes the boundaries of hardware-software co-design. It contributes to local innovation by developing a new solution for a critical real-world application.
- **SDG 10: Reduced Inequalities:** By drastically lowering the cost of advanced prosthetics, this project directly tackles the healthcare inequality faced by low-income populations and marginalized groups, providing access to technology that is typically reserved for the wealthy.

1.7 Detailed Problem Description

1.7.1 Desired Features and Objectives

Based on literature review and stakeholder analysis, the desired features and objectives for the solution can be listed as:

- **Affordability:** Utilize low-cost equipment and sensors.
- **Versatility:** Cover tasks difficult for amputees.

- **Easy to Use:** Require minimal calibration to work and adjusts without difficult procedures.
- **Generalizability:** Work for users with different hands structures and compositions.
- **Scalability:** Can be expanded for future prosthetic design and integration.

1.8 Functional Requirements

The system shall:

- provide visualization of the hand model.
- process EMG signals adequate for inference.
- output angles of joints responsible for hand movement.
- support fast translation of intended movement into motion space.
- cater poses corresponding to concerned joints.
- provide a scalable framework extendable to different subjects.

1.9 Constraints & Limitations

- Limited dataset available and subject-dependence of EMG, necessitate the need for techniques for calibration like domain adaptation.
- FPGA memory constraints of 4.9Mb BRAM & DSP limitations of 220 DSP slices, put a cap on the number of simultaneous MAC operations possible and architecture size[22]
- Quantization accuracy drop (tolerance 10%) require tighter tuning of parameters and quantization aware training.
- Real-time constraints restrict model size as the add delays in the architecture.

- Cost ceiling for hardware components, particularly those for EMG sensors and electrodes should be affordable.
- System not intended for medical certification.

CHAPTER 2

DESIGN CONCEPTS

2.1 Design Alternatives

During the ideation phase, multiple architectural and hardware choices were evaluated to determine the one with the best trade-off between accuracy, latency and power-efficiency.

2.1.1 Morphological Analysis

A morphological chart has been utilized to explore different combinations of subsystems. The selected solution is highlighted in bold in Table 2.1

Function/Sub-system	Option 1	Option 2	Option 3
<i>1. Sensing & Data Acquisition</i>			
Input Modality	Computer (Camera)	Vision Surface (sEMG)	EMG Invasive Implant
Signal Preprocessing	Analog Circuitry (Active)	Digital (PL/FPGA)	Filters Software (PS/ARM)
<i>2. Processing & Intelligence</i>			
Model Architecture	CNN (Spatial)	RNN/LSTM (Temporal)	Transformer (Attention)
Compute Hardware	Microcontroller (STM32)	GPU (Jetson Nano)	SoC FPGA (Zynq-7000)
Numeric Precision	Floating Point (FP32)	Quantized (INT8)	Binary Neural Net (1-bit)
<i>3. System Output & Integration</i>			
Communication	UART / Serial	Bluetooth Low Energy (BLE)	Wi-Fi (MQTT/IoT)
Actuation/Output	Unity 3D (Simulation)	Servo Motors (Prosthetic)	Haptic Feedback
Power Source	Tethered (USB/Wall)	Li-Po (Portable)	Battery Wireless Power Transfer

Table 2.1: Expanded Morphological Chart for sEMG Gesture Recognition System

2.1.2 Discussion of Alternatives

Model Architecture: CNN vs. Transformer

Convolutional Neural Networks (CNNs) were considered due to their efficiency in spatial feature extraction. However, EMG signals are inherently temporal; the gesture interpretation depends upon the firing of signals in sequence over time [23]. While Recurrent Neural Networks (RNNs/LSTMs) can also handle time-series data, they process data sequentially and only work well for small sequences. Thus, they create a latency bottleneck and make parallelization difficult [24]. The Transformer architecture, because of its self-attention mechanism, captures global temporal dependencies more effectively than CNNs and, unlike RNNs, can process sequences in parallel [25, 24]. For FPGA deployment, transformers also avoid the accumulation of quantization errors that RNNs struggle with due to their recurrent nature [26].

Hardware Platform: GPU vs. FPGA

Mobile GPUs (e.g., Nvidia Jetson) offer high throughput but suffer from high power consumption (10W-40W) and fixed architectures. Microcontrollers (e.g., ARM Cortex-M) are low power but lack the compute density for matrix-heavy Transformer inference. Hence, the FPGA is chosen because it allows for:

- **Custom Datapaths:** Hardware can be designed specifically for Matrix-Multiply-Accumulate (MAC) operations.
- **Deterministic Latency:** FPGAs avoid the jitter associated with OS-based scheduling on GPUs.
- **Power Efficiency:** By utilizing quantized integer arithmetic, the FPGA operates within the <5W envelope required for portable prosthetics.

2.2 Metrics for Objectives: Success Criteria

Objective	Metric
Cost-effective	Prototype cost $\leq 10\%$ of commercial alternatives
Accuracy	MAE $< 0.2^\circ$ or accuracy $> 75\%$
Real-time performance	Latency < 80 ms
Scalability	FPGA usage $< 90\%$ enabling cheaper boards

2.3 Critical Design Issues & Mitigation

- **EMG Noise & Variability:** mitigate using filtering, adaptive normalization, and electrode placement guidelines.
- **Model Too Large for FPGA:** reduce complexity via pruning and quantization-aware training.
- **Latency Bottlenecks:** overcome using pipeline parallelism and reduced bit-width operations.
- **Synchronization Issues:** resolve with a fixed data streaming protocol.
- **Visualization Delay:** minimize by employing an optimized communication interface.

CHAPTER 3

DESIGN DETAILS

3.1 Project Focus

Given the scale of this problem, we will be focusing on the aspect that will be most fruitful and fit in our capston timeline. Hence, the focus of this project is restricted to

- **Motion tracking:** developing a model to track continuous hand motion using EMG signals.
- **Visualization:** using a virtual environment to simulate the motion of a hand model in real-time.
- **Hardware acceleration:** deploying the model on a low-cost FPGA for high speed compute requirements.
- **Stimulated EMG:** using stimulated EMG as the input because of its feasibility and low barrier to prototype testing.

This focus of the project excludes a mechanical prosthetic design given its cost and complexity involved, as well as the use of surface EMG sensors, and any medical or clinical approaches due to a lack domain knowledge and time restrictions. The solution only focuses on the engineering aspects that can be developed over an abstraction of the biomechanics and biosignal acquistion.

3.2 Technical Challenge

The main technical task for this capstone project is:

“How can a Transformer-based deep learning model be efficiently designed, quantized, and implemented on a low-cost FPGA to perform real-time, high-accuracy hand movement tracking for a virtual hand from non-invasive sEMG signals?”

The focus is on creating the “*brain*” of the *prosthetic* in the shape of a complete, pipeline that translates EMG signals into accurate, intended gesture movements.

3.3 Technical Requirements

The system shall:

1. **Low-Latency Inference:** preprocess and infer hand pose with latency $\leq 80ms$ [27].
2. **Hand Position Output:** output 10 joint angles that can be used to control complete hand pose using dependent relations with other joints.
3. **Visualization:** provide a real-time visualization at $\geq 30FPS$.
4. **FPGA Resource Usage:** run inference entirely on FPGA within $\leq 90\%$ LUT/BRAM usage to avoid routing congestion.
5. **Power Consumption:** Operate within power consumption $\leq 5W$ [28].
6. **Degrees of Freedom:** Estimate the required number of degrees of freedom (DoF) i.e. 15.
7. **Model Size:** maintain $\leq 250,000$ parameters with 8 – 16 bit quantization, fitting comfortably in the BRAM of the Zedboard FPGA used.
8. **Memory Constraints:** operate within available on-chip memory resources of the Zedboard FPGA (4.9Mb) [22].
9. **Communication Interface:** Support communication with the visualization subsystem via standard protocols such as AXI4.

3.4 Societal Considerations

3.4.1 Healthcare Accessibility

The primary societal driver for this project is the democratization of assistive technology in Pakistan. By utilizing off-the-shelf sensors and a low-cost FPGA (Zedboard

Zynq-7000, comparable to the price of mid-range smartphones) rather than proprietary medical hardware, it lowers the barrier to entry for lower-income background amputees.

3.4.2 Data Privacy (Ethics)

Processing biometric data raises privacy concerns. Unlike cloud-based AI solutions which transmit user data to remote servers, this *Edge AI* approach processes all EMG signals locally, ensuring that sensitive data never leaves the user’s device, adhering to privacy-by-design principles.

3.5 Environment and Sustainability

3.5.1 Power Efficiency

The transition from floating-point processing (typical in GPUs) to fixed-point (INT8) processing on FPGA reduces dynamic power consumption, reducing energy consumption. This also extends battery life, reducing the frequency of charging cycles.

3.5.2 Hardware Longevity

The reconfigurable nature of FPGAs supports sustainability. As the algorithms improves, the hardware can be “upgraded” via a bitstream update rather than requiring to change it all together, reducing electronic waste (e-waste) and allowing for easier R&D.

3.5.3 Reduced Carbon Footprint via Edge Computing

By performing inference locally on the “Edge” (the prosthetic device), this system eliminates the energy costs associated with wireless data transmission and cloud data center processing. Data transmission is often more energy-intensive than local computation.

3.5.4 Algorithmic “Green AI”

The project employs model compression techniques. This adheres to “Green AI” principles by minimizing the FLOPs (Floating Point Operations) required per inference,

ensuring that high-performance gesture recognition does not come at the cost of excessive computational energy.

3.6 Intellectual Contribution

The primary intellectual contribution of this project lies at the intersection of advanced deep learning, biomedical signal processing, and embedded hardware design. It directly addresses a critical gap identified in the literature review.

While existing research has established the limitations of classical machine learning [15] and the performance ceiling of standard CNN/RNN architectures [16], work has only recently begun to explore the use of Transformers for EMG classification. This new research, such as the work by Wei et al. [17] and Park and Lee [18], has been highly promising but has remained almost exclusively in the software domain. These studies have validated the *accuracy* of Transformer models, but have done so using high-power, non-portable hardware (i.e., GPUs) that is unsuitable for a prosthetic device.

Conversely, the field of hardware acceleration for AI has made significant strides in implementing deep learning models on FPGAs [20]. However, as the survey by Li et al. [21] indicates, this work has overwhelmingly focused on accelerating Transformers for Natural Language Processing (NLP) and, to a lesser extent, computer vision.

Our project is novel because it bridges this gap. The intellectual contribution is not simply the creation of another software model, but the design of a complete **hardware-software co-designed pipeline** to take a computationally-demanding, state-of-the-art AI model (Transformer) and deploy it in a real-time, resource-constrained, power-efficient application (EMG-based control).

Specifically, our contributions will be:

1. **A Novel Application of Hardware-Accelerated AI:** We are adding to the body of knowledge by designing one of the first dedicated FPGA accelerators for a *Transformer model specifically tailored for biomedical time-series data (EMG)*. This moves beyond the well-documented NLP domain and requires a new exploration of architectural optimizations.

2. **An Incremental Improvement on Existing Solutions:** Compared to the software-only Transformer models [17, 18], our design will be a significant step forward in terms of *practical viability*. We will provide a blueprint for a system that is not only performant, but also low-power and fast enough for real-world use.
3. **A Supporting Tool for Future Research:** The verified RTL implementation of our Transformer accelerator will serve as a foundational building block (an IP core) for future research in accessible, high-performance assistive devices.

In essence, while previous work has shown that Transformers are *accurate*, our project will demonstrate how to make them *usable and accessible* for a critical health-care application.

CHAPTER 4

DESIGN DETAILS

This chapter will cover the details of the design in software, hardware, and the interfacing between those areas.

4.1 High Level System Design

At a high level, the system takes in EMG signals and outputs a snapshot of the hand movement in the form of angles of selected joints. This pipeline is constituted of three moving parts: the EMG signal generation head which is responsible for providing the signals corresponding to a certain movement; the accelerator, which is the model hosted on the hardware that performs the transformation from the biosignal space to the joint space; and the visualization gateway, which showcases the predicted movement of the hand in a simulated environment to an end user. The figure below 4.1 demonstrates this idea:



Figure 4.1: High-level system architecture linking EMG input, FPGA inference, and Unity visualization.

4.2 Software Subsystem

The software part of the system involves the analysis of sEMG data obtained from Ninapro [29], the signal processing techniques applied on the signals, and the development and training of the model architecture.

4.2.1 Data Analysis

We utilized the Ninapro DB2 [29] dataset for our project. DB2 contains sEMG data acquired using 12 Delsys Trigno electrodes, while kinematic data are acquired using a Cyberglove 2 data glove. The fields of interest in the data are the EMG signals from the 12 channels (from 12 electrodes), and the joint angle voltages corresponding to those signals.

Subject Variation

The sEMG signals as well as the raw voltage values for the joints provided by the glove both vary greatly for different subjects. The raw voltage value difference can be observed from the boxplots provided in Figure 4.2 for 10 joints angle across 40 subjects.

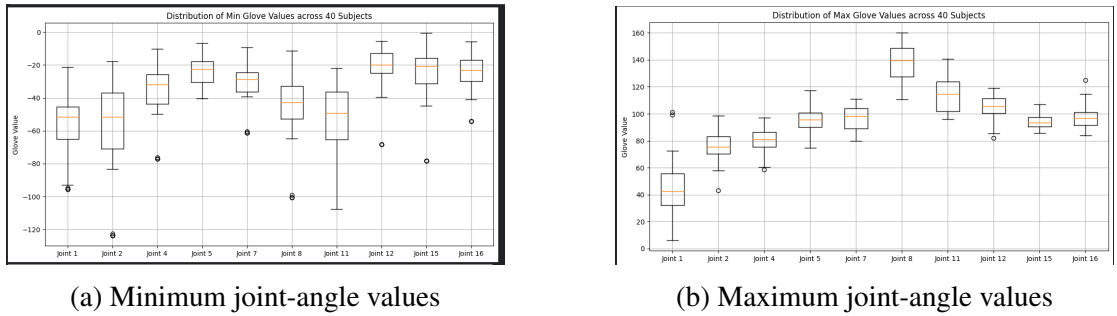


Figure 4.2: Per-subject distribution of representative joint-angle values across the DB2 dataset.

The DB2 dataset contains data from 40 intact subjects (hands). Each subject performed the same experimental protocol under identical conditions. The figure below illustrates the variability in EMG signal amplitude and joint angle range across subjects for a representative gesture, highlighting the inter-subject variability challenge discussed in Section 1.3.3.

To evaluate generalization, we adopted two evaluation protocols:

- **Within-Subject:** The model is trained and tested on data from the same subject using an 80/20 train-test split. This represents the best-case performance and serves as an upper bound.
- **Cross-Subject:** The model is trained on data from $N - 1$ subjects and tested on the held-out subject (LOSO Validation - Leave One Subject Out). This evaluates generalization to unseen users, which is the target deployment scenario.

Exercises

The DB2 dataset is organized into three exercise blocks, each targeting a different category of hand movements [29]:

- **Exercise A (12 gestures):** Introductory training and calibration movements, designed to familiarize subjects with the acquisition setup and ensure consistent start/end postures. These include simple warm-up hand positions and rest phases to establish baseline signals.
- **Exercise B (17 gestures):** Basic finger movements and isometric force patterns, including individual finger flexion/extension and multi-finger combinations. This exercise is most directly relevant to our joint angle estimation task as it produces distinct, isolatable finger movements.
- **Exercise C (23 gestures):** Grasping and functional hand postures, including power grips, pinch grips, and tool-use poses. These involve coordinated multi-joint movements and represent real-world prosthetic use cases.

Each gesture is repeated 6 times per subject in a cue-based paradigm: a visual stimulus prompted the subject to perform the gesture for 5 seconds, followed by a 3-second rest period[29]. For our continuous joint angle estimation task, we used Exercise B and Exercise C, as they provide the richest variation in finger joint angles. Exercise D is

excluded as our model targets the 10 finger joints shown in Figure 4.3 and does not estimate wrist kinematics.

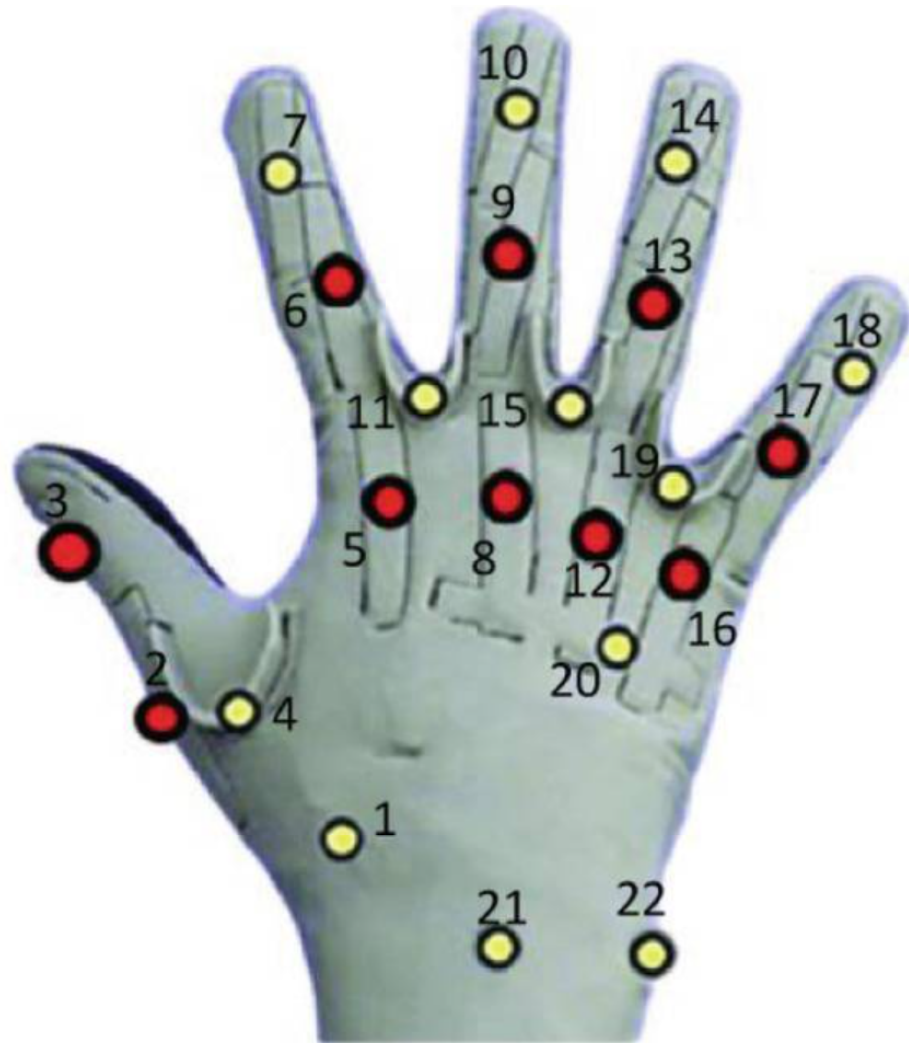


Figure 4.3: 10 joint angles of interest (highlighted in red)

Poses – Joints Analysis

Of the 22 joint angles recorded by the Cyberglove 2, we selected the 10 joints highlighted in Figure 4.3, following the architecture in [30]. These correspond to the Metacarpophalangeal (MCP) and Proximal Interphalangeal (PIP) flexion angles for each of the four fingers, plus the MCP and Interphalangeal (IP) angles of the thumb (shown in the anatomical model of hand in figure 4.4).

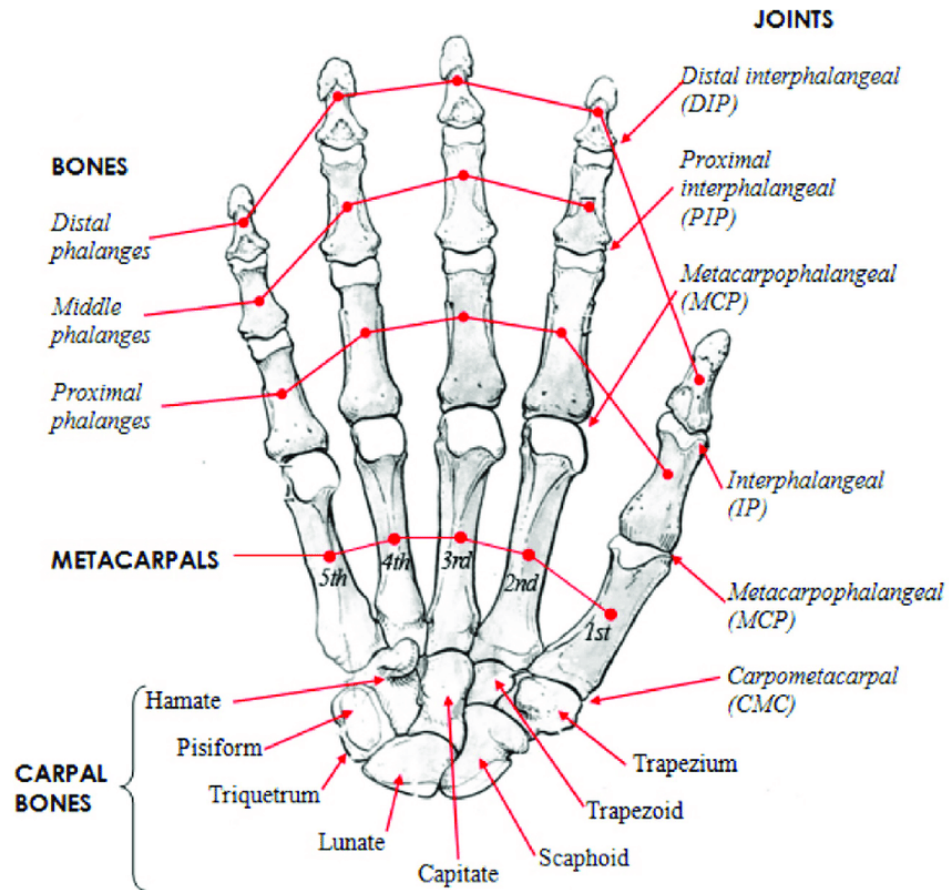


Figure 4.4: Anatomical Structure of Hand

Table 4.1: The 10 target joint angles estimated by the model and their anatomical descriptions.

Index	Joint
5	Index MCP
6	Index PIP
8	Middle MCP
9	Middle PIP
12	Ring MCP
13	Ring PIP
15	Pinky MCP
16	Pinky PIP
2	Thumb MCP
3	Thumb IP

The remaining 12 joints from the Cyberglove (DIP joints, CMC joints, and abduction/adduction angles) are not directly estimated by the model. Instead, they are reconstructed using the biomechanical coupling relations where the idea is that joint

movements in fingers and thumb are coupled and highly interdependent [31, 32]:

$$\theta_{\text{DIP}} \approx \frac{2}{3} \theta_{\text{PIP}} \quad (4.1)$$

$$\theta_{\text{Thumb IP}} \approx \frac{1}{2} \theta_{\text{Thumb MCP}} \quad (4.2)$$

To verify that the selected 10 joints carry sufficient information for reconstructing natural hand poses, we computed the variance explained by these joints across all gestures in Exercises B and C. The selected joints account for the majority of kinematic variance in the dataset, consistent with the principal component analysis of hand synergies reported by Santello et al. [32].

4.2.2 sEMG Signal Conditioning and Preprocessing

Filtering

To ensure high-quality EMG signals, the raw data underwent a multi-stage digital filtering pipeline. First, we applied a fourth-order Butterworth band-pass filter with cutoffs between 5–500 Hz. The magnitude response of the n -th order Butterworth filter is defined as:

$$|H(f)| = \frac{1}{\sqrt{1 + \left(\frac{f}{f_c}\right)^{2n}}} \quad (4.3)$$

where f_c represents the cutoff frequency and $n = 4$. This removes low-frequency movement artifacts and high-frequency noise while preserving the relevant physiological muscle activity spectrum.

To suppress power line interference at 50 Hz and its associated harmonics, a recursive IIR comb notch filter was employed. The transfer function for the comb filter is given by:

$$H(z) = G \frac{1 - z^{-N}}{1 - \rho^N z^{-N}} \quad (4.4)$$

where $N = f_s/f_0$ (with $f_s = 2000$ Hz and $f_0 = 50$ Hz) determines the notch spacing, and ρ is related to the quality factor $Q = 30$. This configuration achieves narrow rejection bands, effectively isolating interference while minimizing phase distortion of

the underlying EMG signal.

Normalization

We employed μ -law companding to normalize the EMG signals. The transformation is defined as:

$$f(x) = \frac{\text{sign}(x) \cdot \ln(1 + \mu|x|)}{\ln(1 + \mu)}, \quad \mu = 255,$$

where x is the input normalized to $[-1, 1]$. This nonlinear mapping compresses large amplitude values while expanding small ones, effectively enhancing the dynamic range of low-intensity signals without saturating high-intensity activations. In practice, this improves robustness to inter-subject variability and highlights subtle muscle activations that are critical for fine motor control.

Windowing

Data from each exercise repetition is segmented into overlapping windows of length $400ms$ samples with a hop size of $100ms$ samples prior to training, following the windowing strategy described in Section 3.6.2.

In addition to these steps, rectification is also applied which takes the absolute value of the signal to convert bipolar signals to unipolar form so that model can focus on magnitude of the signals rather than its oscillatory patterns.

$$f(x) = |x|$$

4.2.3 Model Architecture and Training

Architecture

The proposed model follows a hybrid architecture combining temporal feature extraction with attention-based sequence modeling. The input sEMG signal $\mathbf{X} \in \mathbb{R}^{L \times C}$ (where $L = 200$ and $C = 12$) is first processed by a 1D Convolutional layer (ker-

nel size $k = 3$, stride $s = 1$, padding $p = 1$) to extract local temporal correlations and project the input into the model dimension $d_{\text{model}} = 128$.

The sequence is then encoded using Rotary Positional Embeddings (RoPE) before entering a Transformer backbone consisting of two encoder blocks. Each block utilizes Multi-Head Self-Attention (MHSA) with $h = 8$ heads and a feed-forward network ($d_{ff} = 256$) with ReLU activation, followed by layer normalization and residual connections. The final representation is passed through a linear output head to regress 10 joint angles representing the hand kinematics. The total parameter count is approximately 80,000, which fits within the BRAM constraints of the Zynq-7000 FPGA [22].

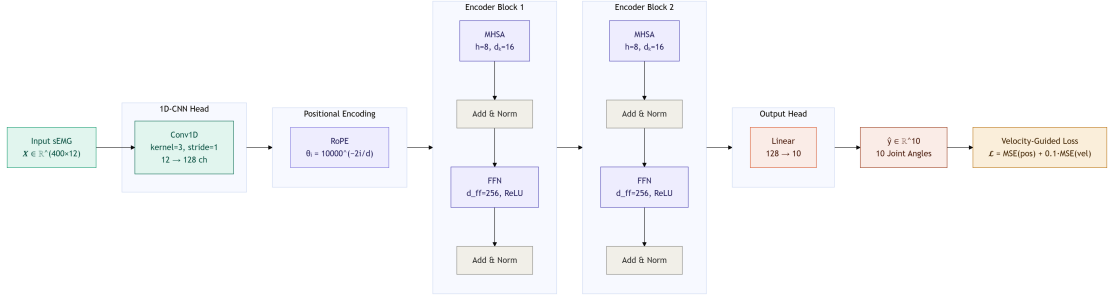


Figure 4.5: RoFormer model architecture: 1D-CNN input head, RoPE positional encoding, two transformer encoder blocks with residual connections, and a linear output head regressing 10 joint angles. The velocity-guided loss supervises both position and velocity during training.

RoPE Embeddings

Unlike standard absolute positional encodings, Rotary Positional Embedding (RoPE) [30] encodes relative positional information by rotating the Query (q) and Key (k) vectors in the complex plane. For a vector at position m , the transformation is defined as:

$$\mathbf{q}_m = \mathbf{R}_{\Theta, m}^d \mathbf{W}_q \mathbf{x}_m, \quad \mathbf{k}_n = \mathbf{R}_{\Theta, n}^d \mathbf{W}_k \mathbf{x}_n \quad (4.5)$$

where $\mathbf{R}_{\Theta, m}^d$ is a block-diagonal rotation matrix parameterized by frequencies $\Theta = \{\theta_i = 10000^{-2i/d} \mid i = 1, \dots, d/2\}$. The uniqueness of RoPE lies in the fact that the attention score depends only on the relative distance $m - n$:

$$\langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle = \mathbf{g}(\mathbf{x}_m, \mathbf{x}_n, m - n) \quad (4.6)$$

This property ensures that attention naturally decays with increasing temporal distance, which is well-suited for capturing the short-burst dynamics of muscle contractions while still retaining long-range context across the 200-sample window. Furthermore, RoPE avoids the need for a separate learned positional embedding table, reducing parameter count and simplifying quantization for FPGA deployment.

Velocity-Guided Loss

To ensure predicted joint angles follow biologically plausible trajectories, we employ a custom Velocity-Guided Loss function. Standard MSE supervision operates only on static positional error, which can produce predictions that are accurate in magnitude but discontinuous between frames, manifesting as jitter in the visualization. Our loss function addresses this by jointly penalizing positional error and the discrepancy between predicted and ground-truth joint velocities:

$$\mathcal{L}_{\text{total}} = \underbrace{\text{MSE}(y_{\text{pred}}, y_{\text{curr}})}_{\text{position term}} + \lambda_v \cdot \underbrace{\text{MSE}(\dot{y}_{\text{pred}}, \dot{y}_{\text{true}})}_{\text{velocity term}} \quad (4.7)$$

where the velocity terms are computed as finite differences over consecutive windows:

$$\dot{y}_{\text{true}} = y_{\text{curr}} - y_{\text{prev}} \quad (4.8)$$

$$\dot{y}_{\text{pred}} = y_{\text{pred}} - y_{\text{prev}} \quad (4.9)$$

and $\lambda_v = 0.1$ is a regularization weight determined empirically. The velocity term forces the model to learn the *intent* of the movement rather than just static positions, producing smoother output kinematics that translate directly to more natural hand animation in the Unity visualization.

Quantization

We performed Quantization-Aware Training (QAT) to prepare the model for integer-only inference on the FPGA. Using PyTorch’s `torch.ao.quantization` API, fake quantization nodes were inserted into the computation graph during training, sim-

ulating the rounding and clipping effects of 8-bit fixed-point arithmetic. This converts `float32` weights and activations to `int8`, yielding a $4\times$ reduction in model memory footprint (from ≈ 320 kB to ≈ 80 kB) and enabling the use of integer DSP slices on the Zynq-7000 rather than floating-point units.

The QAT procedure followed the standard observer-based calibration workflow:

1. Train the full-precision (`float32`) model to convergence.
2. Insert `FakeQuantize` observers on all weights and activations.
3. Fine-tune for an additional 10 epochs with a reduced learning rate (1×10^{-5}) to allow the model to adapt to quantization noise.
4. Freeze observer statistics and convert to a fully quantized `int8` model via `torch.ao.quantization.convert()`.

The tolerance for accuracy degradation due to quantization was set at 10% relative drop in R^2 score, consistent with the constraint in Section 2.2. Activation functions were replaced with ReLU prior to quantization, as GeLU’s non-polynomial form is not efficiently representable in fixed-point arithmetic.

Training Configuration

The model was trained on a local workstation (CPU-only) using the hyperparameters in Table 4.2, derived from a grid search over learning rate, dropout, and window length.

Table 4.2: Model hyperparameters and training configuration.

Parameter	Value
Window length (L)	200 samples (100 ms)
Hop size	40 samples (20 ms)
Model dimension (d_{model})	128
Attention heads (h)	8
Encoder layers	2
Feed-forward dim (d_{ff})	256
Batch size	32
Learning rate	1×10^{-4}
QAT fine-tune LR	1×10^{-5}
Max epochs (FP32)	50
Max epochs (QAT)	10
Early stopping patience	6 epochs
Dropout	0.1
λ_v (velocity loss)	0.1

Training used the Adam optimizer with default momentum parameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$). A `ReduceLROnPlateau` scheduler halved the learning rate after 3 epochs of no improvement on validation MSE, preventing oscillation near convergence. Data was split per-subject into 80% training and 20% validation, with no data leakage between windows belonging to the same gesture repetition.

4.3 Hardware Design and Acceleration

The hardware implementation leverages the Zynq-7000 SoC architecture to provide deterministic, low-latency execution of the Transformer-based kinematic regressor. By offloading the computationally intensive attention mechanisms to the Programmable Logic (PL), the system achieves real-time performance that exceeds sequential CPU-based execution.

4.3.1 Design using High-Level Synthesis (HLS)

The Transformer architecture is developed in C++ and synthesized into Register-Transfer Level (RTL) logic using Vitis HLS E. To optimize the design for the Zedboard’s hardware constraints, several optimization strategies are employed:

Numerical Precision

To balance computational throughput with resource utilization, the design utilizes fixed-point arithmetic `ap_fixed<8, 3>` (3 integer bits, 5 fractional bits) for internal activations, with `ap_fixed <16, 6>` for housing any intermediate products or accumulations, which are then scaled back to the original representation E . This choice provides sufficient dynamic range for EMG-based features while significantly reducing DSP slice consumption compared to 32-bit, and even 16-bit floating-point units. The conversion from floating-point weights to quantized hardware values is performed using the formula:

$$Q = \text{round}(F \cdot 2^{BW-IW})$$

where F is the original float, BW is the total bit-width, and IW is the integer bit-width.

Parallelism and Pipelining

Hardware pragmas such as `#pragma HLS PIPELINE` are applied to the main processing loops to achieve a target Initiation Interval (II) of 1. To maximize the data throughput of the AXI4 interfaces, the design implements an 8-lane parallel data bus using `#pragma HLS ARRAY_PARTITION`[Ref: HLS Optimization/Schedule Report]. This allows the hardware to process feature dimensions concurrently, by accessing multiple BRAM (Block RAM) units, effectively matching the bandwidth of the underlying 64-bit AXI interconnect. Multi-head attention is parallelized by unrolling the head-dimension using the `#pragma HLS UNROLL` directive, enabling simultaneous calculation of attention scores across independent heads.

To minimize the consumption of on-chip Block RAM (BRAM) and prevent a linear increase in resource usage with model depth, the encoder architecture is designed for iterative layer reuse. This is achieved through a ping-pong buffering strategy, where two high-speed internal memory banks (`buffer_seq0` and `buffer_seq1`) interchangeably serve as source and destination for successive sub-layers.

Each buffer is sized to the maximum dimensionality of the feature space, defined

as $SEQ_LEN \times D_MODEL$. Here, SEQ_LEN corresponds to the temporal length of the EMG sliding window, while D_MODEL represents the width of the embedding space. During a forward pass, the output of one functional block (e.g., Multi-Head Attention) is stored in one buffer, which then serves as the immutable input for the subsequent block (e.g., Feed-Forward Network). This temporal multiplexing allows the implementation of an arbitrarily deep Transformer block using a fixed hardware footprint, ensuring the design remains within the physical resource constraints of the Zedboard's Zynq-7000 fabric.

Memory Management

Rotary Positional Embeddings (RoPE) are implemented using a 2D pre-computed Look-Up Table (LUT) stored in Block RAM (BRAM). By utilizing a 2D array structure ($LUT[SEQ_LEN][HALF_DIM]$), the HLS compiler performs precise polyhedral analysis, ensuring stall-free access to trigonometric coefficients. The weights of the model are offloaded to the DDR memory to preserve on-chip BRAM for activations. These weights are accessed via the `m_axi` interface using burst-mode transfers to minimize memory-access latency. To further optimize resource usage, a time-multiplexed memory reuse strategy is employed, where `shared_w` and `shared_b` buffers are repurposed between the Attention and Feed-Forward layers (refer to Appendix E for more details).

4.3.2 Vivado Block Design and AXI Interfacing

The synthesized IP block is integrated into the broader system-on-chip ecosystem using the Vivado IP Integrator. The design establishes a high-speed data path between the Processing System (PS) and the Programmable Logic (PL).

AXI4-Lite Interface

AXI Lite Interface which reads and writes to a select memory mapped registers, is used for the Control and Status Register (CSR) interface. This allows the ARM Cortex-A9

core to trigger the start of inference, monitor the ‘idle/ready’ states, and update model hyperparameters dynamically (refer to Appendix E for more details).

AXI4 Full (Memory Mapped)

Three independent high-performance (HP) slave ports on the Zynq Processing System are dedicated to the IP’s master AXI interfaces (gmem0, gmem1, and gmem2). To prevent AXI crossbar arbitration bottlenecks and address space overlaps, three separate AXI SmartConnect instances are utilized. This architecture provides a dedicated "private lane" for weights, input features, and output results, ensuring that the Transformer IP can saturate the DDR bandwidth without interference from the ARM CPU’s instruction fetches.

Clocking Strategy

The PL logic operates at a clock frequency of 100MHz, synchronized with the PS7 FCLK_CLK0 to ensure timing closure and stable data arbitration across the AXI Interconnect.

4.4 Communication and Integration Layer

This section describes the integration of the hardware-accelerated model with the external visualization environment.

4.4.1 UART Communication

The FPGA sends data over to the Processing Side using AXI, and then the ARM core forwards the data over UART. Considering the close range of communication from chip and the wired nature of the protocol, UART is suitable for sending and receiving data. It is used at a fixed baud rate of 115200.

GUI to Unity Communication

The User Datagram Protocol (UDP) is used over to send received output joint angles from the FPGA to Unity, to minimize overhead, considering its speed over alternatives like TCP.

Data Serialization

Upon completion of an FPGA inference cycle, a cache invalidation command (`Xil_DCacheInvalidateRange`) is issued by the ARM processor to ensure the most recent joint angle predictions are read directly from DDR, bypassing stale cache lines. The resulting fixed-point joint angles are cast into 32-bit floats and serialized before transmission to the Unity environment for more details.

Asynchronous Transmission

The stack is configured in **Interrupt-driven mode** to ensure that the network overhead does not stall the kinematic decoding pipeline. This allows the ARM processor to remain in a low-power "Wait for Interrupt" (WFI) state or perform housekeeping tasks while the FPGA handles the heavy-duty numerical regression.

4.4.2 Unity XR Hand Integration

The final stage of the integration involves a custom C# receiver within the Unity environment. The Unity 3D environment with XR hands package [33] is used for visualizing the joint angle prediction linked to the model output via UDP sockets from the GUI. The binary payload is unpacked into a floating-point array corresponding to the joint space. These predicted joint angles are mapped to the local rotation transforms of the h Hand's virtual bone hierarchy (see Appendix E for more details).

4.4.3 GUI Architecture and Visualization Pipeline

The graphical interface serves a dual purpose: it acts as the control front-end for gesture selection and dataset streaming, and as the real-time visualization back-end for both

ground-truth and FPGA-predicted hand kinematics. The full pipeline is implemented across two tightly coupled components: 1) a Unity 3D application on the host PC and, 2) the embedded inference engine on the ZedBoard, communicating over UART via UDP.

Gesture Selection Interface

On launch, the Unity application presents the user with a gesture selection panel. A dropdown menu enumerates the available gesture classes drawn from the pre-stored Ninapro DB2 dataset (Exercises A, B, and C), allowing the user to select one or more target gestures for demonstration. Each entry in the dropdown corresponds to a labelled gesture class in the dataset (e.g. index finger flexion, power grip, pinch), with the full list dynamically populated at runtime by reading the gesture label manifest from the dataset directory.

Upon confirmation, the application retrieves the pre-processed sEMG windows associated with the selected gesture and queues them for sequential streaming. This design decouples gesture selection from the inference pipeline, allowing the user to explore individual gesture classes in isolation or chain multiple gestures into a continuous demonstration sequence without interrupting the running system.

Dual-Window Visualization Layout

The Unity scene is partitioned into two side-by-side viewports, each rendering an independent skeletal hand model:

- **Ground-truth window (left):** Animates the actual hand motion corresponding to the selected gesture, driven directly by the Cyberglove joint angles stored in the Ninapro DB2 dataset. This serves as the reference trajectory against which the model's predictions are visually compared.
- **Predicted window (right):** Animates the hand pose inferred by the FPGA. The 10 joint angles returned by the hardware inference engine are received over

UART, passed through the post-processing pipeline (Kalman smoothing and anatomical constraint clamping, Section 4.4.4), and applied to the skeletal rig via Unity’s XR Hands.[33].

Rendering both windows simultaneously allows for direct qualitative assessment of model fidelity: the user can observe in real time how closely the predicted hand motion tracks the ground-truth kinematics, and identify any joints where the model under- or over-predicts flexion.

4.4.4 Post-Processing: Joint Angle Smoothing

The raw joint angle predictions output by the RoFormer model exhibit frame-to-frame jitter, a consequence of independent per-window inference with no temporal continuity enforced between consecutive output frames. While the velocity-guided loss partially mitigates this during training, a dedicated post-processing smoothing stage is applied at inference time to produce output kinematics suitable for real-time visualization and prosthetic control. Two variants of the Kalman filter are evaluated for this purpose.

Standard Kalman Filter

The standard Kalman filter models each joint angle as a scalar state evolving under a constant-velocity assumption. For each of the 10 joints independently, the filter maintains a state estimate $\hat{x}_t$ and an uncertainty estimate P_t , updated at every inference step according to the following two-step recursion.

Prediction step. The prior uncertainty is propagated forward in time:

$$P_t^- = P_{t-1} + Q \quad (4.10)$$

where Q is the *process noise covariance*, controlling how much the true joint angle is assumed to change between frames. A higher Q allows the filter to track fast movements but passes more noise through to the output.

Update step. The incoming model prediction z_t (the measurement) is fused with the prior estimate using the Kalman gain K_t :

$$K_t = \frac{P_t^-}{P_t^- + R} \quad (4.11)$$

$$\hat{x}_t = \hat{x}_{t-1} + K_t (z_t - \hat{x}_{t-1}) \quad (4.12)$$

$$P_t = (1 - K_t) P_t^- \quad (4.13)$$

where R is the *measurement noise covariance*, representing the assumed noise level of the model’s raw predictions. The Kalman gain $K_t \in (0, 1)$ interpolates between the prior estimate and the new measurement: when $R \gg Q$, the filter trusts its own prediction more, yielding a smoother but laggier output; when $Q \gg R$, the filter follows the measurements closely, trading smoothness for responsiveness.

The filter is applied independently to all 10 joints in parallel. The initial state $\hat{x}_0$ is set to the first predicted angle for each joint, and the initial uncertainty $P_0 = 1.0$. The hyperparameters used are summarized in Table 4.3.

Table 4.3: Standard Kalman filter hyperparameters.

Parameter	Symbol	Value
Process noise covariance	Q	1×10^{-4}
Measurement noise covariance	R	0.05
Initial uncertainty	P_0	1.0

The primary limitation of the standard Kalman filter in this context is that Q and R are fixed scalars, set once before inference. This means the filter applies identical smoothing regardless of whether the hand is stationary (where heavy smoothing is desirable) or executing a fast transition between gestures (where the filter’s lag becomes detrimental to tracking accuracy and visual responsiveness).

Adaptive Kalman Filter

To address the fixed-noise limitation of the standard filter, we also evaluated an Adaptive Kalman Filter (AKF) that adjusts the measurement noise covariance R_t dynamically at

each timestep based on the observed innovation (the residual between the incoming measurement and the prior prediction):

$$\epsilon_t = z_t - \hat{x}_{t-1} \quad (4.14)$$

The measurement noise is updated using an exponentially weighted moving average (EWMA) of the squared innovation:

$$R_t = (1 - \alpha) R_{t-1} + \alpha \epsilon_t^2 \quad (4.15)$$

where $\alpha \in (0, 1)$ is the adaptation rate. When the innovation is large (i.e. the model's prediction deviates sharply from the smoothed estimate, indicating a genuine fast movement), R_t increases, which lowers the Kalman gain and reduces smoothing to allow the filter to track the transition. When the innovation is small (stationary or slow motion), R_t decreases, increasing smoothing and suppressing noise. The prediction and update steps remain identical to the standard filter, with R replaced by R_t .

Table 4.4: Adaptive Kalman filter hyperparameters.

Parameter	Symbol	Value
Process noise covariance	Q	1×10^{-4}
Initial measurement noise	R_0	0.05
Adaptation rate (EWMA)	α	0.1
Initial uncertainty	P_0	1.0

The key advantage of the AKF is that it behaves like a heavy smoother during periods of slow movement and automatically relaxes to a more responsive filter during fast transitions, without requiring manual re-tuning of R for different subjects or gesture speeds.

CHAPTER 5

PROTOTYPING AND TESTING

This chapter presents the prototype implementation of the hardware-software co-designed system described in 4, along with the testing methodologies, KPIs, and results obtained from the physical ZedBoard-based platform.

5.1 Prototype Overview

The prototype integrates three physical components: the ZedBoard Zynq-7000 SoC, running the quantized RoFormer inference engine on its Programmable Logic (PL) fabric and communicating via UART from its ARM Processing System (PS), a host PC running the Unity 3D visualization environment, and a pre-recorded sEMG dataset from the Ninapro DB2 benchmark streamed as input in lieu of live electrodes.

5.2 Testing Methodology

Testing is structured in three stages: subsystem-level verification, integration testing, and end-to-end system evaluation. Each stage is designed to validate a specific set of functional and technical requirements from Section 1.8 and Section 3.3, respectively.

5.2.1 Model Evaluation and Performance Metrics

The performance of the kinematic regressor is quantitatively evaluated using a multi-metric approach. While standard Deep Learning models often rely on loss values alone, the assessment of a prosthetic control system requires metrics that correlate directly with user-perceived fluidity and anatomical accuracy. The primary metrics utilized are the Coefficient of Determination (R^2) and the Root Mean Square Error (RMSE), calculated across all the 10 predicted joint angle values.

Coefficient of Determination (R^2)

The R^2 score is employed to measure the proportion of variance in the hand's joint angles that is predictable from the sEMG input signals. In the context of this project, a high R^2 score indicates that the Transformer core has successfully captured the underlying biomechanical intent rather than merely memorizing the dataset. It is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (5.1)$$

where y_i represents the ground-truth joint angle value, $\hat{y}_i$ is the predicted value from the hardware-accelerated model, and $\bar{y}$ is the mean of the observed data.

Root Mean Square Error (RMSE)

To provide a measure of absolute error in degrees or radians, the RMSE is implemented. This metric is particularly sensitive to outliers, making it ideal for identifying "stuttering" or erratic predictions that would negatively impact the user experience. The RMSE is calculated as:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (5.2)$$

5.2.2 FPGA Transformer Inference Verification

The quantized INT8 RoFormer implementation on the PL fabric is validated against the software (FP32 PyTorch) baseline using identical Ninapro DB2 test windows.

- **Accuracy Comparison:** The R^2 score and Mean Absolute Error (MAE) of the FPGA output are compared against the software model on the same held-out test set.
- **Quantization Error:** Per-joint angular deviation between FP32 and INT8 outputs is measured to confirm the quantization error remained within the 10% tol-

erance specified in Section 3.3.

- **Resource Utilisation:** LUT, BRAM, and DSP slice usage is recorded from the Vivado implementation report.

5.2.3 Visualization Pipeline Verification

The Unity visualization subsystem is tested independently by sending pre-defined angle sequences over UDP and confirming correct skeletal rendering.

Frame Rate Test

Unity's frame counter is logged over a 60-second window to verify sustained ≥ 30 FPS rendering.

Packet Loss Test

UDP packet counts are compared between the sender (PS) and receiver (Unity) over 1000 consecutive frames.

5.3 Key Performance Indicators (KPIs)

Table 5.1 lists the KPIs used to evaluate prototype performance against the design targets established in Section 3.3.

Table 5.1: KPIs and design targets for prototype evaluation.

KPI	Metric	Design Target	Source
Inference Accuracy	R^2 / MAE (degrees)	$R^2 \geq 0.75$ and $MAE < 0.2^\circ$	Section 3.3
End-to-End Latency	ms	≤ 80 ms	[27]
FPGA LUT Usage	% of available LUTs	$\leq 80\%$	Section 3.3
FPGA BRAM Usage	% of 4.9 Mb	$\leq 80\%$	[22]
Power Consumption	Watts	≤ 5 W	[28]
Visualization Frame Rate	FPS	≥ 30 FPS	Section 3.3
Calibration Time	Minutes	< 5 min	Section 3.3

5.4 Results

5.4.1 Inference Accuracy

Table 5.2 compares the joint-wise MAE of the FPGA INT8 model against the software FP32 baseline on the Ninapro DB2 test set.

Table 5.2: Per-joint RMSE — software FP32 vs. FPGA INT8.

Joint	FP32 RMSE	INT8 RMSE	Deviation (%)
MCP Index	0.019	0.046	142.1%
PIP Index	0.021	0.040	90.5%
MCP Middle	0.029	0.033	13.8%
PIP Middle	0.028	0.030	7.1%
MCP Ring	0.044	0.211	379.5%
PIP Ring	0.026	0.053	103.8%
MCP Pinky	0.054	0.134	148.1%
PIP Pinky	0.030	0.053	76.7%
MCP Thumb	0.049	0.134	173.5%
IP Thumb	0.031	0.001	-96.8%
Mean	0.033	0.068	106.1%

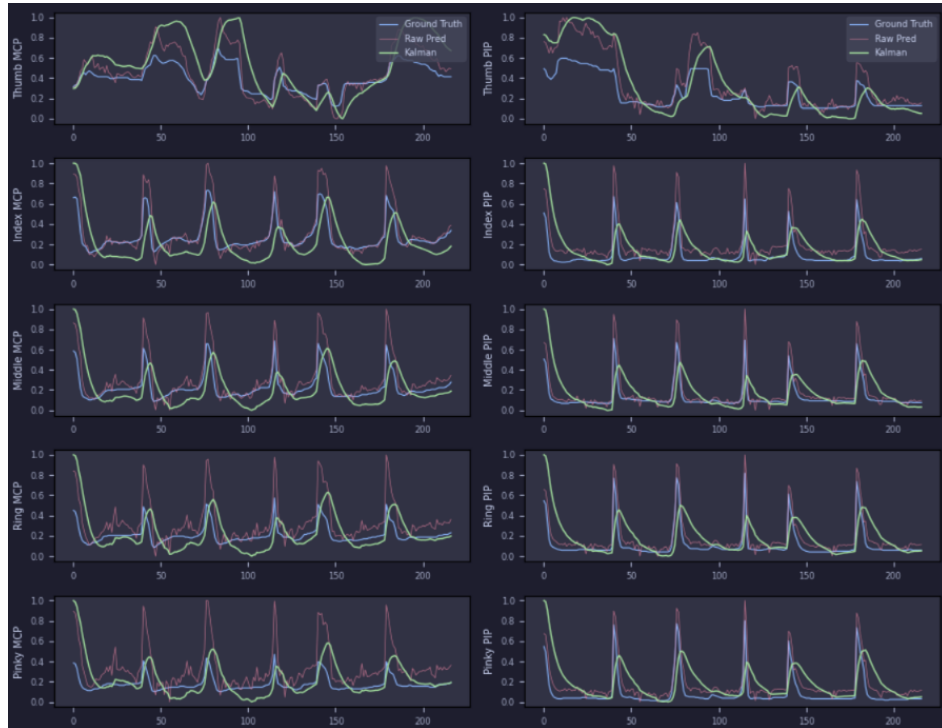


Figure 5.1: Predicted vs. ground-truth joint angles for a representative 200 ms test window (FPGA INT8 output).

5.4.2 FPGA Resource Utilization and Power

The table 5.3 breaks down the utilization required to fit our mixed precision quantized model.

Table 5.3: FPGA resource utilization (Vivado implementation report).

Resource	Used	Available	Utilization (%)
LUTs	13,983	53,200	35
FFs	14,900	106,400	14
BRAM	268	280	95
DSP Slices	220	220	100

5.4.3 Post-processing Ablation

Surprisingly, although both the Kalman and Adaptive Kalman filters successfully smoothed the joint angle predictions to better reflect natural hand motion, they resulted in a lower overall R^2 score—dropping from 0.89 (raw) to 0.82 (post-processed). Visual analysis revealed the cause: while the raw model output occasionally exhibited large deviations from the ground truth, it typically self-corrected in the subsequent inference cycle. In contrast, the smoothing mechanism of the Kalman filters introduced a temporal lag, causing them to take longer to realign with the ground truth after a sudden spike. Consequently, the raw, albeit jittery, predictions quantitatively outperformed the post-processed results.

5.4.4 Visualization Performance

The Unity visualization sustained an average frame rate of 60 FPS over the 60-second test window. UDP packet loss is 2% across 1000 frames.

Appendices

APPENDIX A

MATERIAL AND COMPONENT DETAILS

Table A.1 lists all hardware components used in the prototype .

Table A.1: Bill of Materials (BOM) for prototype.

Component	Vendor	Qty	Cost (USD)
ZedBoard Evaluation Kit	Digilent	1	–
Ethernet Cable (Cat5e)	–	1	1 –
Total Prototype Cost (USD)			–

A.1 Datasheet References

- Zynq-7000 SoC Data Sheet: Overview [22]. Available at: <https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview>
- Zedboard: <https://digilent.com/reference/programmable-logic/zedboard/start>

A.2 System Requirements

A.2.1 Hardware Requirements

- ZedBoard Evaluation Kit
- Host PC (Windows 10/11, ≥ 8 GB RAM)
- Ethernet cable (Cat5e or higher with RJ45 connector) for UDP communication
- Micro-USB cable for UART/JTAG programming (preferred) or SD Card
- 12V DC power supply for ZedBoard

A.2.2 Software Requirements

- AMD Vivado Design Suite (version ≥ 2024.1) — for bitstream generation
- AMD Vitis (version ≥ 2024.1) — for ARM Cortex-A9 (PS) firmware
- Python with PyTorch – for model training and QAT
- Unity 3D with XR Hands package – for visualization

APPENDIX B

PROJECT SCHEDULE

This appendix presents the full project schedule spanning both Capstone 1 (August 2025 – December 2025) and Capstone 2 (January 2026 – May 2026). The schedule is organised by major work package, reflecting the iterative nature of the hardware–software co-design methodology adopted by the team.

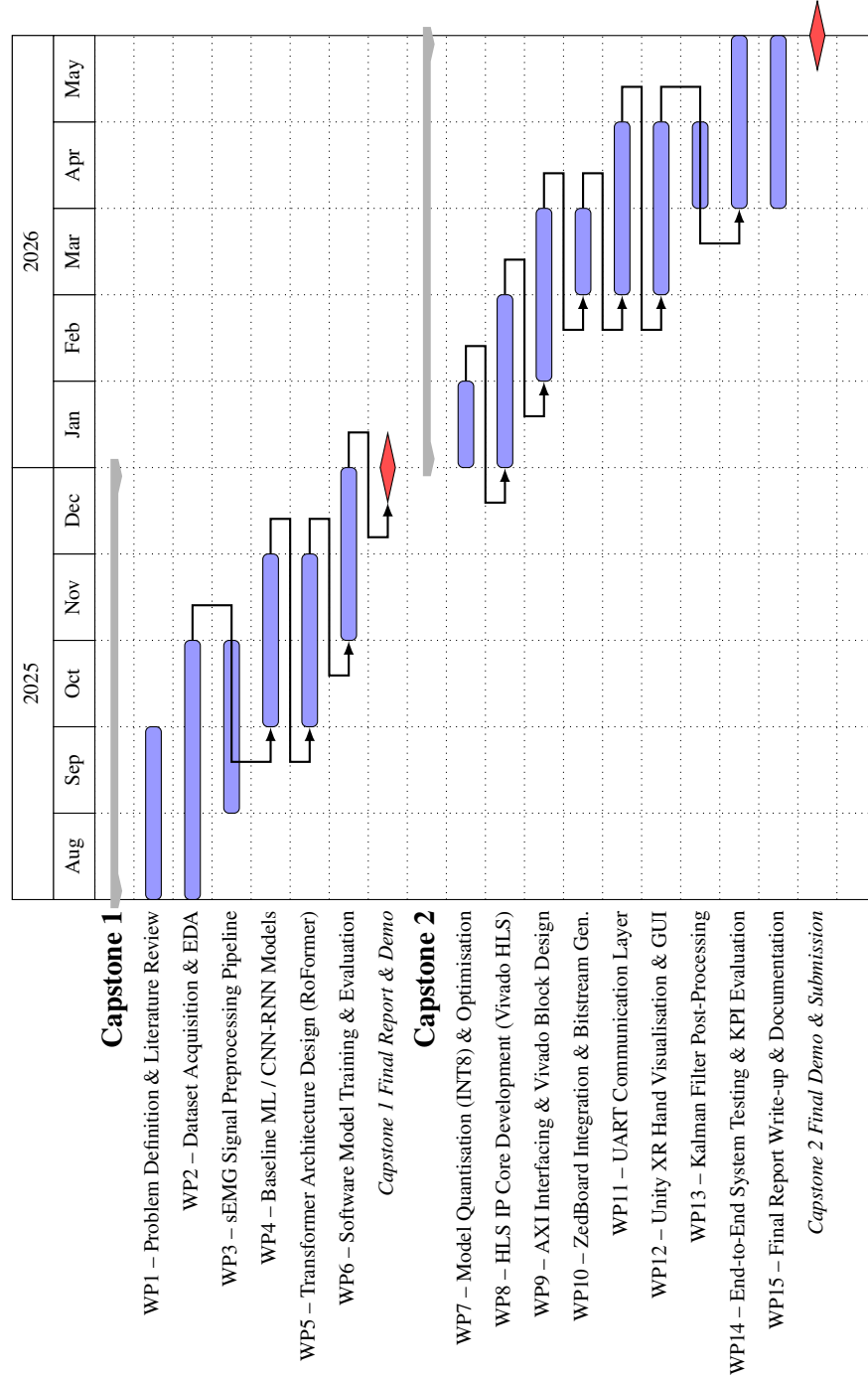


Figure B.1: Project Gantt chart covering Capstone 1 (Aug 2025 – Dec 2025) and Capstone 2 (Jan 2026 – May 2026). Red diamonds mark key milestones; blue bars indicate planned work packages.

Work Package Summary

The following table summarises each work package (WP), its primary deliverable, and the responsible team members.

Table B.1: Work package summary.

WP	Title	Key Deliverable	Lead(s)
1	Problem Definition & Literature Review	Annotated bibliography; problem statement	All
2	Dataset Acquisition & Exploratory Data Analysis	Cleaned NinaPro DB2 dataset; EDA report	Quddusi Kashaf
3	sEMG Signal Preprocessing Pipeline	Windowing, normalisation, and feature scripts	Quddusi Kashaf
4	Baseline ML / CNN-RNN Models	Benchmark accuracy figures	Abdullah Amir
5	Transformer Architecture Design (RoFormer)	Model specification document	Abdullah Amir
6	Software Model Training & Evaluation	Trained FP32 model; per-joint RMSE report	Abdullah Amir, Quddusi Kashaf
7	Model Quantisation (INT8) & Optimisation	Quantised model; accuracy vs. resource trade-off report	Abdullah Amir
8	HLS IP Core Development	Synthesisable HLS kernel; co-simulation results	Saif Nazir, Abdullah Amir
9	AXI Interfacing & Vivado Block Design	Verified block design; timing closure report	Saif Nazir
10	ZedBoard Integration & Bitstream Generation	Programmed ZedBoard; live inference demo	Saif Nazir
11	UART Communication Layer	Serial protocol implementation; latency measurements	Saif Nazir, Quddusi Kashaf
12	Unity XR Hand Visualisation & GUI	Real-time animated hand; GUI application	Saif Nazir
13	Kalman Filter Post-Processing	Smoothed joint-angle output; ablation study	Quddusi Kashaf
14	End-to-End System Testing & KPI Evaluation	KPI report; FPGA resource utilisation table	All
15	Final Report Write-up & Documentation	Capstone 2 report; repository README	All

APPENDIX C

HARDWARE SETUP AND OPERATING PROCEDURE

This appendix provides sufficient detail to recreate the prototype. It covers the system setup, hardware components, software environment, and operational procedures for the hardware-software co-designed EMG-based hand tracking system.

C.1 ZedBoard Physical Setup

1. Connect 12 V power supply to barrel jack (J20).
2. Connect the USB-UART port of ZedBoard (J14) which is labeled UART to a PC using the MicroUSB cable.
3. Insert the 4GB SD card included with ZedBoard into the SD card slot (J12) located on the underside of ZedBoard PCB. This SD card comes preloaded with demo software and contains a basic Linux configuration used to implement the demos listed in the later sections.
4. Set the ZedBoard boot mode jumpers (JP7–JP11) to JTAG mode for initial programming, or SD card mode for standalone operation.

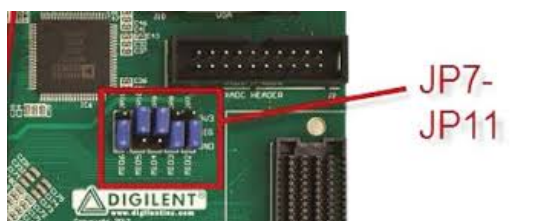


Figure C.1: ZedBoard jumper configuration for SD card boot mode.

5. Turn power switch (SW8) to the ON position. ZedBoard will power on and the Green Power Good LED (LD13) should illuminate.

C.2 Bitstream Generation and Programming

C.2.1 Generating the Bitstream in Vivado

1. Open Vivado and load the project file:

`/fpga/vivado_project/emg_transformer.xpr`

2. In the *Flow Navigator*, click **Generate Bitstream**.

3. Once complete, export the hardware platform:

File → **Export** → **Export Hardware** (include bitstream).

C.2.2 Programming the ZedBoard

1. Open AMD Vitis and create a new platform project using the exported `.xsa` hardware description file.
2. Build and flash the ARM Cortex-A9 firmware via JTAG.
3. Alternatively, copy `BOOT.BIN` to an SD card root and boot in SD card mode for standalone operation.

C.3 Running the Full Pipeline

1. Power on the ZedBoard and wait for the image to boot (≈ 30 seconds; confirmed via UART terminal (and physically by the orange LED on the board near the UART port) and the blue LED on the board).
2. Run the pipeline in the GUI for your selected Gesture, and the FPGA will start processing by taking input data and sending output back, indicated by the red LEDs LED0, LED1, switching on and off.
3. Press **Play** in Unity on the host PC and send values from the GUI, the hand model should begin animating within 1–2 seconds.
4. To stop, press `Ctrl+C` on the ZedBoard terminal and stop Play mode in Unity.

C.4 Troubleshooting

Table C.1: Common issues and resolutions.

Symptom	Likely Cause	Resolution
ZedBoard does not boot	Incorrect jumper settings	Verify JP7–JP11 per Section C
Bitstream fails to load	Vivado version mismatch	Regenerate with correct Vivado version
Jerky hand animation	Kalman filter not running	Verify <code>emg_inference</code> is active on PS

APPENDIX D

BLOCK DESIGN AND SCHEMATICS

This appendix documents the Vivado IP Integrator block design used to assemble the Zynq-7000 SoC platform and the AXI address map assigned to the RoFormer accelerator. Together, these views fully specify the hardware integration layer that connects the custom HLS IP to the ARM Processing System (PS) and DDR memory.

D.1 Vivado Block Design

Figure D.1 shows the complete Vivado IP Integrator block design for the RoFormer inference platform. The design contains four top-level IP blocks:

- **RoFormer_Top_0** – The custom HLS-synthesised accelerator IP. It exposes one AXI-Lite slave port (`s_axi_control`) for start/done register access by the PS, and two AXI master ports (`m_axi_gmem0` for reading the input EMG window and `m_axi_gmem1` for writing the predicted joint angles) that burst-transfer data directly to and from DDR.
- **ZYNQ7 Processing System (processing_system7_0)** – The ARM Cortex-A9 dual-core PS. Three high-performance AXI slave ports (`S_AXI_HP0–HP2`) accept burst traffic from the interconnect, providing the DDR bandwidth required for large activation transfers. The `M_AXI_GP0` general-purpose master port initiates control transactions to the HLS IP.
- **AXI Interconnect (axi_mem_intercon)** – Routes the two AXI master ports of the HLS IP (`S00_AXI`, `S01_AXI`, `S02_AXI`) to the three HP slave ports of the PS (`M00_AXI–M02_AXI`), enabling simultaneous read and write bursts without bus contention.
- **AXI SmartConnect (axi_smc)** – Bridges the PS `M_AXI_GP0` master through to the AXI-Lite control interface of the HLS IP and the AXI GPIO peripheral.

- **AXI GPIO (axi_gpio_0)** – Drives the eight on-board LEDs (leds_8bits) to provide visual status indication during inference (see Section 4.4 of the main report).

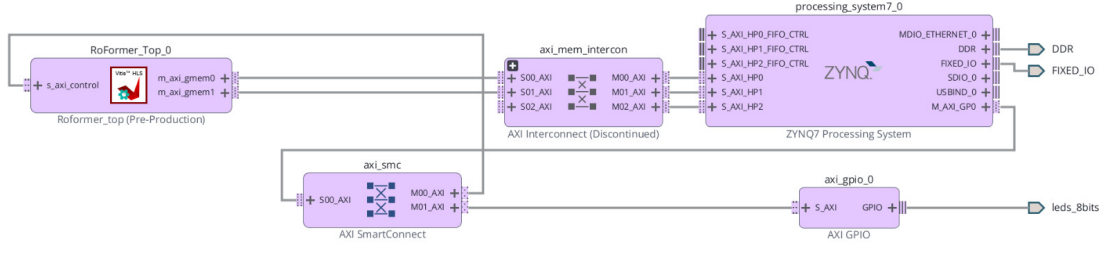


Figure D.1: Vivado IP Integrator block design. The RoFormer_Top_0 HLS accelerator (left) connects to the ZYNQ7 Processing System (right) via a three-slave AXI Interconnect for high-bandwidth DDR access and an AXI SmartConnect for low-latency control register access. The AXI GPIO peripheral drives the status LEDs on the ZedBoard.

D.2 AXI Address Map

Figure D.2 shows the Vivado Address Editor view for Network 0, which governs the memory-mapped address space visible to the RoFormer_Top_0 master ports. The HLS IP's two AXI master ports (m_axi_gmem0 and m_axi_gmem1) can both access the full 384 MB window (0x0000_0000–0x17FF_FFFF) divided across three 128 MB Processing System slave segments, as summarised in Table D.1.

Table D.1: AXI master address assignments for the RoFormer_Top_0 IP (Network 0). Both m_axi_gmem0 and m_axi_gmem1 share the same address map.

Slave Segment	Base Address	High Address	Size
PS S_AXI_HP0 (DDR segment 0)	0x0000_0000	0x07FF_FFFF	128 MB
PS S_AXI_HP1 (DDR segment 1)	0x0800_0000	0x0FFF_FFFF	128 MB
PS S_AXI_HP2 (DDR segment 2)	0x1000_0000	0x17FF_FFFF	128 MB

The firmware DDR buffers defined in `main.c` (DDR_UART_RX_ADDR = 0x0A00_0000, DDR_FPGA_IN_ADDR = 0x0A01_0000, DDR_FPGA_OUT_ADDR = 0x0C00_0000,

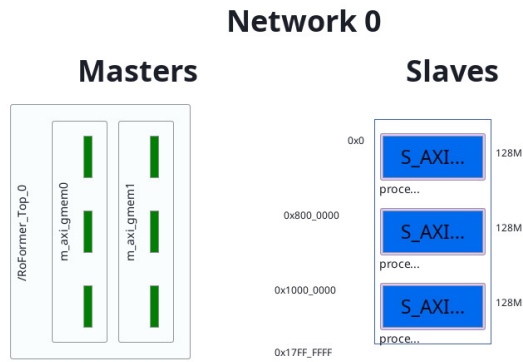


Figure D.2: Vivado Address Editor – Network 0. Both AXI master ports of the RoFormer HLS IP are mapped to three contiguous 128 MB DDR segments through the PS high-performance slave ports, giving the accelerator direct burst access to the full inference data region defined in `main.c`.

DDR_UART_TX_ADDR = 0x0C01_0000) all fall within the HP1 and HP2 segments respectively, confirming that every ARM-to-FPGA and FPGA-to-ARM transfer remains within the mapped address window.

APPENDIX E

SELECTED SOURCE CODE

This appendix presents the key source files for the two main components of the system: the ARM Cortex-A9 firmware running on the Zynq PS (written in C), and the software RoFormer model implemented in PyTorch. Full source code is available in the project repository.

F.1 Selected Code Listings

F.1.1 ARM Firmware – main.c

The firmware orchestrates the full real-time inference pipeline on the Zynq-7000 Processing System (PS). It receives a raw floating-point EMG window over UART, converts it to the `ap_fixed<10, 3>` format expected by the FPGA accelerator, triggers inference, reads back the fixed-point joint-angle predictions, converts them to floats, and transmits the results to the host PC.

Listing E.1: ARM firmware (main.c) – UART-driven real-time inference loop.

```
1 #include "xparameters.h"
2 #include "xil_printf.h"
3 #include "xil_cache.h"
4 #include "xroformer_top.h" // HLS-generated driver for the
   RoFormer IP
5 #include "xgpio.h"
6 #include "xuartps.h"
7 #include "xscugic.h"
8 #include "platform.h"
9 #include <math.h>
10 #include <stdint.h>
11 #include <stdio.h>
12
```

```

13 // -----
14 // Memory Map (safe, heap-independent DDR regions)
15 // -----
16 #define DDR_UART_RX_ADDR 0x0A000000 // Raw float32 EMG window from
    PC
17 #define DDR_FPGA_IN_ADDR 0x0A010000 // Fixed-point window fed to
    FPGA
18 #define DDR_FPGA_OUT_ADDR 0x0C000000 // Fixed-point predictions
    from FPGA
19 #define DDR_UART_TX_ADDR 0x0C010000 // Float32 predictions sent
    back to PC
20
21 // -----
22 // Fixed-Point Format: ap_fixed<10, 3> (7 fractional bits)
23 // -----
24 #define TOTAL_BITS 10
25 #define INT_BITS 3
26 #define FRAC_BITS (TOTAL_BITS - INT_BITS) // = 7 => scale
    factor = 128
27
28 /** Convert a float to ap_fixed<10,3> (signed 10-bit, 7 fractional
    bits).
29 * Values outside [-4, ~3.99] are clamped to prevent overflow. */
30 int16_t float_to_fixed(float val) {
31     float scaled = val * (1 << FRAC_BITS); // Scale: multiply by
        128
32     if (scaled > 511.0f) scaled = 511.0f; // Clamp to 10-bit
        signed max
33     if (scaled < -512.0f) scaled = -512.0f; // Clamp to 10-bit
        signed min
34     return (int16_t)roundf(scaled); // Round to nearest
        integer
35 }
36
37 /** Convert ap_fixed<10,3> back to float.

```

```

38  * The upper 6 bits are padding; sign-extend from bit 9 before
    dividing. */
39 float fixed_to_float(int16_t val) {
40     if (val & 0x0200)          // If sign bit (bit 9) is set, value is
        negative
41     val |= 0xFC00;            // Sign-extend: set upper 6 padding bits
        to 1
42     return (float)val / (float)(1 << FRAC_BITS);
43 }
44
45 // -----
46 // Peripheral IDs
47 // -----
48 #define GPIO_ID                XPAR_AXI_GPIO_0_BASEADDR
49 #define ROFORMER_ID            XPAR_ROFORMER_TOP_0_BASEADDR
50 #define UART_DEVICE_ID         XPAR_UART1_BASEADDR
51 #define INTC_DEVICE_ID         XPAR_XSCUGIC_0_BASEADDR
52 #define UART_INT_IRQ_ID        82
53
54 // -----
55 // Data Sizes
56 // -----
57 #define INPUT_FLOATS            2400                // 12 channels x 200
        time-steps
58 #define OUTPUT_FLOATS          10                // 10 joint angles
59 #define EXPECTED_BYTES (INPUT_FLOATS * sizeof(float))
60 #define OUTPUT_BYTES           (OUTPUT_FLOATS * sizeof(float))
61
62 // -----
63 // LED Status Indicators
64 // -----
65 #define LED_OFF                 0x00
66 #define LED_SENDING 0x01 // LED0: receiving / processing
67 #define LED_DONE                0x02 // LED1: inference complete
68

```



```

69 // -----
70 //  Global Peripheral Instances & State Flags
71 // -----
72 XRoformer_top    Model;
73 XUartPs          Uart_Ps;
74 XScuGic          InterruptController;
75
76 volatile int packet_ready_flag    = 0;
77 volatile u32 actual_bytes_received = 0;
78
79 // -----
80 //  UART Interrupt Service Routine
81 // -----
82 /** Called by the UART driver when a receive event occurs.
83  *   Two events are handled:
84  *   RECV_DATA : the requested byte count was received exactly.
85  *   RECV_TOUT : the line went quiet mid-packet (partial receive).
86  */
87 void UartInterruptHandler(void *CallBackRef, u32 Event, unsigned int
88   EventData) {
89     if (Event == XUARTPS_EVENT_RECV_DATA) {
90         actual_bytes_received = EventData;
91         packet_ready_flag = 1;
92     } else if (Event == XUARTPS_EVENT_RECV_TOUT) {
93         // Ignore phantom timeout interrupts that carry no data
94         if (EventData > 0) {
95             actual_bytes_received = EventData;
96             packet_ready_flag = 1;
97         }
98     }
99 }
100 // -----
101 //  Interrupt Controller Initialisation
102 // -----

```

```

102 int SetupInterruptSystem() {
103     XScuGic_Config *IntcConfig =
104         XScuGic_LookupConfig(INTC_DEVICE_ID);
105
106     if (!IntcConfig) return XST_FAILURE;
107
108     if (XScuGic_CfgInitialize(&InterruptController, IntcConfig,
109                               IntcConfig->CpuBaseAddress) !=
110         XST_SUCCESS)
111         return XST_FAILURE;
112
113     // Register the GIC handler with the ARM exception table
114     Xil_ExceptionInit();
115     Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_INT,
116                                 (Xil_ExceptionHandler)XScuGic_InterruptHandler,
117                                 &InterruptController);
118
119     // Wire the UART IRQ to the smart high-level driver handler
120     if (XScuGic_Connect(&InterruptController, UART_INT_IRQ_ID,
121                        (Xil_ExceptionHandler)XUartPs_InterruptHandler,
122                        (void *)&Uart_Ps) != XST_SUCCESS)
123         return XST_FAILURE;
124
125     // Register our application callback
126     XUartPs_SetHandler(&Uart_Ps,
127                       (XUartPs_Handler)UartInterruptHandler, &Uart_Ps);
128
129     // Enable RX FIFO threshold (RXOVR) and idle-line timeout (TOUT)
130     // interrupts
131     XUartPs_SetInterruptMask(&Uart_Ps, XUARTPS_IXR_RXOVR |
132                             XUARTPS_IXR_TOUT);
133
134     XScuGic_Enable(&InterruptController, UART_INT_IRQ_ID);
135     Xil_ExceptionEnable();
136     return XST_SUCCESS;
137 }

```

```

132
133 // -----
134 //  Main Entry Point
135 // -----
136 int main() {
137     init_platform();
138     xil_printf("\n=====\\r\\n");
139     xil_printf("          BOOTING SYSTEM\\r\\n");
140     xil_printf("=====\\r\\n");
141
142     // --- GPIO (LEDs) ---
143     XGpio Gpio;
144     XGpio_Config *cfg_ptr = XGpio_LookupConfig(GPIO_ID);
145     XGpio_CfgInitialize(&Gpio, cfg_ptr, cfg_ptr->BaseAddress);
146     XGpio_SetDataDirection(&Gpio, 1, 0x00); // All LED pins as
        output
147     XGpio_DiscreteWrite(&Gpio, 1, LED_OFF);
148
149     // --- UART ---
150     XUartPs_Config *UartConfig =
        XUartPs_LookupConfig(UART_DEVICE_ID);
151     XUartPs_CfgInitialize(&Uart_Ps, UartConfig,
        UartConfig->BaseAddress);
152     XUartPs_SetBaudRate(&Uart_Ps, 115200);
153     XUartPs_SetFifoThreshold(&Uart_Ps, 32); // Interrupt every 32
        bytes
154     XUartPs_SetOperMode(&Uart_Ps, XUARTPS_OPER_MODE_NORMAL);
155
156     if (SetupInterruptSystem() != XST_SUCCESS) {
157         xil_printf("[FATAL] Interrupt setup failed.\\r\\n");
158         return XST_FAILURE;
159     }
160     XUartPs_SetRecvTimeout(&Uart_Ps, 8); // 8-character idle timeout
161
162     // --- RoFormer HLS IP ---

```

```

163 xil_printf("Initializing RoFormer IP...\r\n");
164 XRoformer_top_Config *RofCfg =
    XRoformer_top_LookupConfig(ROFORMER_ID);
165 XRoformer_top_CfgInitialize(&Model, RofCfg);
166 XRoformer_top_Set_input_axi(&Model, DDR_FPGA_IN_ADDR); // AXI
    master read
167 XRoformer_top_Set_output_axi(&Model, DDR_FPGA_OUT_ADDR); // AXI
    master write
168
169 xil_printf("System ready. Waiting for %d bytes...\r\n",
    EXPECTED_BYTES);
170
171 // Prime the DMA-less UART receiver
172 XUartPs_Recv(&Uart_Ps, (u8*)DDR_UART_RX_ADDR, EXPECTED_BYTES);
173
174 //
    -----
175 // Main Inference Loop
176 //
    -----
177 while (1) {
178     if (packet_ready_flag == 1) {
179
180         if (actual_bytes_received == EXPECTED_BYTES) {
181             XGpio_DiscreteWrite(&Gpio, 1, LED_SENDING);
182
183             // Step 1: Convert received float32 EMG data to
                fixed-point
184             float *uart_rx = (float *)DDR_UART_RX_ADDR;
185             short *fpga_in = (short *)DDR_FPGA_IN_ADDR;
186             for (int i = 0; i < INPUT_FLOATS; i++)
187                 fpga_in[i] = float_to_fixed(uart_rx[i]);
188
189             // Flush D-cache so the FPGA AXI master sees fresh
                data

```

```

190     Xil_DCacheFlushRange (DDR_FPGA_IN_ADDR,
191                           INPUT_FLOATS * sizeof(short));
192
193     // Step 2: Trigger FPGA inference and wait for
194     completion
195     XRoformer_top_Start (&Model);
196
197     while (!XRoformer_top_IsDone (&Model));
198
199     // Invalidate D-cache so CPU reads fresh FPGA output
200     Xil_DCacheInvalidateRange (DDR_FPGA_OUT_ADDR,
201                                OUTPUT_FLOATS *
202                                sizeof(short));
203
204     XGpio_DiscreteWrite (&Gpio, 1, LED_DONE);
205
206     // Step 3: Convert fixed-point predictions back to
207     float32
208     short *fpga_out = (short *)DDR_FPGA_OUT_ADDR;
209     float *uart_tx  = (float *)DDR_UART_TX_ADDR;
210     for (int i = 0; i < OUTPUT_FLOATS; i++)
211         uart_tx[i] = fixed_to_float (fpga_out[i]);
212
213     // Step 4: Transmit 10 float32 joint angles to host
214     PC
215     XUartPs_Send (&Uart_Ps, (u8*)uart_tx, OUTPUT_BYTES);
216
217     }
218
219     // Reset flag and re-arm receiver for the next packet
220     packet_ready_flag = 0;
221     XUartPs_Recv (&Uart_Ps, (u8*)DDR_UART_RX_ADDR,
222                  EXPECTED_BYTES);
223
224     }
225
226     }
227
228     return 0;
229 }

```

F.1.2 RoFormer Model – `roformer.py`

The software model is a lightweight Transformer variant that replaces conventional absolute positional encodings with Rotary Position Embeddings (RoPE), which encode relative position information directly into the query–key dot-product. A 1D-CNN input head first projects the 12-channel sEMG signal into the model dimension, after which two RoFormer encoder layers extract temporal dependencies before a linear head regresses 10 joint angles.

Listing E.2: RoFormer model definition (`roformer.py`).

```
1 import torch
2 import torch.nn as nn
3 import torch.quantization as quant
4
5
6 # -----
7 # Rotary Position Embedding (RoPE)
8 # -----
9
10 class RotaryEmbedding(nn.Module):
11     """Pre-computes cosine/sine rotation matrices for RoPE.
12
13     For each position  $t$  and each frequency pair  $(\cos, \sin)$  are
14     cached up to
15     ``max_position_embeddings``. The cache is extended on-the-fly
16     if a
17     longer sequence is encountered at runtime.
18     """
19
20     def __init__(self, dim, max_position_embeddings=2048, base=10000,
21                 device=None):
22         super().__init__()
23         self.dim = dim
24         self.base = base
```

```

24     # Inverse frequencies: theta_i = 1 / (base^(2i/d))
25     inv_freq = 1.0 / (base ** (torch.arange(0, dim,
26                                     2).float().to(device)
27                                     / dim))
28
29     self.register_buffer("inv_freq", inv_freq)
30
31     # Pre-build cache so torch.jit.trace works without dynamic
32     # control flow
33
34     self._set_cos_sin_cache(seq_len=max_position_embeddings,
35                             device=self.inv_freq.device,
36                             dtype=torch.get_default_dtype())
37
38     def _set_cos_sin_cache(self, seq_len, device, dtype):
39         """Build and cache the (seq_len, dim) cos/sin rotation
40         matrices."""
41         self.max_seq_len_cached = seq_len
42         t = torch.arange(seq_len, device=device,
43                           dtype=self.inv_freq.dtype)
44         freqs = torch.einsum("i,j->ij", t, self.inv_freq) #
45                 [seq_len, dim/2]
46
47         # Concatenate to cover all dim dimensions: [seq_len, dim]
48         emb = torch.cat((freqs, freqs), dim=-1)
49         self.register_buffer("cos_cached", emb.cos().to(dtype),
50                             persistent=False)
51         self.register_buffer("sin_cached", emb.sin().to(dtype),
52                             persistent=False)
53
54     def forward(self, x, seq_len=None):
55         """Return cos and sin slices of length ``seq_len``.
56
57         Args:
58             x: Query or key tensor (used only for device/dtype
59                context).
60             seq_len: Sequence length of the current input.
61
62         Returns:

```

```

51         Tuple of (cos, sin), each shaped [seq_len, dim].
52     """
53     if seq_len > self.max_seq_len_cached:
54         self._set_cos_sin_cache(seq_len, device=x.device,
55                                 dtype=x.dtype)
56     return (self.cos_cached[:seq_len].to(dtype=x.dtype),
57            self.sin_cached[:seq_len].to(dtype=x.dtype))
58
59 def rotate_half(x):
60     """Split x along the last dim and rotate: [-x2, x1].
61
62     This implements the half-rotation used in RoPE to produce
63     orthogonal
64     sine/cosine components without a full matrix multiply.
65     """
66     x1 = x[..., : x.shape[-1] // 2]
67     x2 = x[..., x.shape[-1] // 2 :]
68     return torch.cat((-x2, x1), dim=-1)
69
70 def apply_rotary_pos_emb(q, k, cos, sin):
71     """Apply RoPE to query and key tensors.
72
73     Args:
74         q, k: Query/key tensors, shape [batch, seq_len, d_model].
75         cos, sin: Rotation matrices, shape [seq_len, d_model].
76
77     Returns:
78         Rotated (q_embed, k_embed) of the same shapes.
79     """
80     cos = cos.unsqueeze(0) # Broadcast over batch: [1, seq_len,
81                            # d_model]
82     sin = sin.unsqueeze(0)
83     q_embed = (q * cos) + (rotate_half(q) * sin)
84     k_embed = (k * cos) + (rotate_half(k) * sin)

```



```

83     return q_embed, k_embed
84
85
86     # -----
87     # RoFormer Encoder Layer
88     # -----
89
90     class RoFormerLayer(nn.Module):
91         """Single RoFormer encoder block.
92
93         Identical to a standard Transformer encoder layer except that
94         RoPE
95         is applied to queries and keys before scaled dot-product
96         attention,
97         replacing the conventional absolute positional encoding.
98         """
99
100         def __init__(self, d_model, nhead, dim_feedforward=512,
101                     dropout=0.1):
102             super().__init__()
103             self.self_attn = nn.MultiheadAttention(d_model, nhead,
104                                                     dropout=dropout,
105                                                     batch_first=True)
106
107             # Feed-forward sub-layer
108             self.linear1 = nn.Linear(d_model, dim_feedforward)
109             self.linear2 = nn.Linear(dim_feedforward, d_model)
110             self.activation = nn.ReLU()
111             self.dropout = nn.Dropout(dropout)
112             self.dropout1 = nn.Dropout(dropout)
113             self.dropout2 = nn.Dropout(dropout)
114
115             # Pre-norm (applied after residual add, i.e., Post-LN
116             # convention)
117             self.norm1 = nn.LayerNorm(d_model)
118             self.norm2 = nn.LayerNorm(d_model)

```

```

114 def forward(self, src, cos=None, sin=None,
115             src_mask=None, src_key_padding_mask=None):
116     """Forward pass with optional RoPE application.
117
118     Args:
119         src: Input tensor [batch, seq_len, d_model].
120         cos, sin: Pre-computed RoPE matrices [seq_len, d_model].
121     Returns:
122         Output tensor [batch, seq_len, d_model].
123     """
124     q = k = src
125     # Apply rotary embeddings to queries and keys if provided
126     if cos is not None and sin is not None:
127         q, k = apply_rotary_pos_emb(q, k, cos, sin)
128
129     # Multi-head self-attention with residual connection +
130     # LayerNorm
131     attn_out = self.self_attn(q, k, src,
132                               attn_mask=src_mask,
133                               key_padding_mask=src_key_padding_mask)[0]
134     src = self.norm1(src + self.dropout1(attn_out))
135
136     # Position-wise feed-forward with residual connection +
137     # LayerNorm
138     ff_out =
139         self.linear2(self.dropout(self.activation(self.linear1(src))))
140     src = self.norm2(src + self.dropout2(ff_out))
141
142     return src
143
144 # -----
145 # Full RoFormer Model
146 # -----

```

```

146 class RoFormer(nn.Module):
147     """Lightweight RoFormer for continuous sEMG-to-joint-angle
148         regression.
149
150     Architecture:
151         1. Conv1D input head -- projects 12 sEMG channels to d_model
152         2. RoPE module -- pre-computes position rotation
153             matrices
154         3. N x RoFormerLayer -- Transformer encoder with rotary
155             attention
156         4. Global average pool over time
157         5. Linear output head -- regresses ``num_angles`` joint
158             angles
159
160     Args:
161         num_channels (int): Number of sEMG input channels (default:
162             12).
163         time_steps (int): Window length in samples (default: 200).
164         num_layers (int): Number of stacked encoder layers
165             (default: 4).
166         num_heads (int): Number of attention heads (default: 8).
167         d_model (int): Internal model dimension (default: 128).
168         d_ff (int): Feed-forward hidden dimension (default:
169             512).
170         num_angles (int): Number of output joint angles (default:
171             10).
172         dropout (float): Dropout probability (default: 0.1).
173
174     """
175
176     def __init__(self, num_channels=12, time_steps=200, num_layers=4,
177                 num_heads=8, d_model=128, d_ff=512,
178                 num_angles=10, dropout=0.1):
179         super().__init__()
180
181         # Input projection: (batch, channels, time) -> (batch,
182             d_model, time)

```

```

172     self.conv1d = nn.Conv1d(num_channels, d_model,
173                               kernel_size=3, padding=1)
174     self.ropes = RotaryEmbedding(d_model)
175     self.layers = nn.ModuleList(
176         [RoFormerLayer(d_model, num_heads, d_ff, dropout)
177          for _ in range(num_layers)]
178     )
179     self.fc = nn.Linear(d_model, num_angles)
180     self.dropout = nn.Dropout(dropout)
181
182     def forward(self, x):
183         """Forward pass.
184
185         Args:
186             x: EMG window tensor, shape [batch, num_channels,
187              time_steps].
188
189         Returns:
190             Predicted joint angles, shape [batch, num_angles].
191         """
192         x = self.conv1d(x)          # [batch, d_model, time_steps]
193         x = x.transpose(1, 2)      # [batch, time_steps, d_model]
194         x = self.dropout(x)
195
196         seq_len = x.shape[1]
197         cos, sin = self.ropes(x, seq_len=seq_len)
198
199         for layer in self.layers:
200             x = layer(x, cos, sin)
201
202         x = x.mean(dim=1)          # Global average pool over time
203                                   axis
204         return self.fc(x)         # [batch, num_angles]

```

```

205 # Quantisation-Aware Training (QAT) Wrapper
206 # -----
207
208 class QuantizedRoFormer(nn.Module):
209     """Wraps RoFormer with PyTorch QAT stubs for INT8 quantisation.
210
211     QuantStub / DeQuantStub bracket the floating-point sub-model so
212     that
213
214     the quantisation calibration and fake-quantisation passes applied
215     during QAT training correctly simulate INT8 inference precision.
216     This wrapper is used during QAT training only; for deployment the
217     model is converted with ``torch.quantization.convert()``.
218
219     """
220
221     def __init__(self, model_fp32):
222         super().__init__()
223
224         self.quant = quant.QuantStub() # Insert fake-quant
225             at input
226         self.dequant = quant.DeQuantStub() # Remove fake-quant
227             at output
228         self.model_fp32 = model_fp32
229
230     def forward(self, x):
231         x = self.quant(x) # Simulate INT8 quantisation of input
232         x = self.model_fp32(x)
233         x = self.dequant(x) # Restore float32 domain at output
234         return x

```

F.2.3 HLS Inference Kernel – RoFormer_Top.cpp

The HLS kernel implements the full RoFormer inference pipeline as a single top-level function synthesised into a custom AXI-interfaced IP core. It exposes two AXI master ports (gmem0 for input, gmem1 for output) and one AXI-Lite slave port for control registers, conforming to the Vivado IP Integrator conventions. A *ping-pong* activation

buffer strategy (`act_A / act_B`) eliminates on-chip BRAM conflicts between consecutive layers without requiring explicit synchronisation barriers. Six pipeline stages are executed sequentially: (1) AXI input read with zero-padding, (2) Conv1D projection, (3) Multi-Head Attention with RoPE and Layer Normalisation, (4) Feed-Forward Network with Layer Normalisation, (5) Global average pooling, and (6) Linear output head.

Listing E.3: HLS top-level kernel (`RoFormer_Top.cpp`). Each layer reads from one ping-pong buffer and writes to the other, avoiding RAW hazards while keeping activations on-chip.

```

1  #include "Conv1dLayer.h"
2  #include "FCLayer.h"
3  #include "FFNorm.h"
4  #include "Pooling.h"
5  #include "MHANorm.h"
6  #include "types.h"      // data_t: ap_fixed<10,3>
7  #include "config.h"     // IN_CHANNELS=12, D_MODEL=128, SEQ_LEN=200,
                           OUT_DIM=10
8  #include "../weights/fc_weights.h" // Statically linked FC weight
                           arrays
9
10 // -----
11 // Top-Level Function
12 // AXI-M read:  input_axi  [IN_CHANNELS * SEQ_LEN]  =  12 x 200 =
                           2400
13 // AXI-M write: output_axi [OUT_DIM]                =  10 joint
                           angles
14 // -----
15 void RoFormer_Top(
16     const data_t input_axi[], // Raw fixed-point EMG window from
                           DDR
17     data_t output_axi[]      // Predicted joint angles written to
                           DDR
18 )

```

```

19 {
20     // AXI master interfaces -- separate bundles prevent port
        conflicts
21 #pragma HLS INTERFACE mode=m_axi port=input_axi bundle=gmem0 \
22         depth=2400 offset=slave
23 #pragma HLS INTERFACE mode=m_axi port=output_axi bundle=gmem1 \
24         depth=2400 offset=slave
25
26     // AXI-Lite slave: exposes base-address registers to the PS
27 #pragma HLS INTERFACE mode=s_axilite port=input_axi
28 #pragma HLS INTERFACE mode=s_axilite port=output_axi
29 #pragma HLS INTERFACE mode=s_axilite port=return
30
31     //
        -----
32     // Ping-Pong Activation Buffers
33     // Layers alternate: A->B, B->A, A->B ...
34     // Cyclic partitioning (factor=8) allows 8 parallel reads/writes
35     // per cycle, matching the unrolled PIPELINE II=1 loops below.
36     //
        -----
37     data_t act_A[D_MODEL * SEQ_LEN];
38     data_t act_B[D_MODEL * SEQ_LEN];
39 #pragma HLS ARRAY_PARTITION variable=act_A cyclic factor=8 dim=1
40 #pragma HLS ARRAY_PARTITION variable=act_B cyclic factor=8 dim=1
41
42     // FC weight/bias arrays are fully partitioned so the linear
        layer
43     // can access all coefficients in a single clock cycle
44 #pragma HLS ARRAY_PARTITION variable=fc_w_flat complete dim=1
45 #pragma HLS ARRAY_PARTITION variable=fc_b_flat complete dim=1
46
47     //
        -----
48     // Stage 1: AXI Input Read

```

```

49 // Copy the 2400-element fixed-point input window into act_A.
50 // The remainder of act_A (D_MODEL*SEQ_LEN - IN_CHANNELS*SEQ_LEN
51 // elements) is zero-padded so Conv1D sees a clean D_MODEL-wide
52 // buffer without uninitialized memory.
53 //
-----
54 READ_INPUT_VALID:
55     for (int i = 0; i < IN_CHANNELS * SEQ_LEN; ++i) {
56 #pragma HLS PIPELINE II=1
57         act_A[i] = input_axi[i];
58     }
59
60 READ_INPUT_PAD:
61     for (int i = IN_CHANNELS * SEQ_LEN; i < D_MODEL * SEQ_LEN; ++i) {
62 #pragma HLS PIPELINE II=1
63         act_A[i] = 0;
64     }
65
66 //
-----
67 // Stage 2: Conv1D Projection (act_A -> act_B)
68 // Projects IN_CHANNELS=12 to D_MODEL=128 using a kernel-size-3
69 // convolution with same-padding, producing [SEQ_LEN, D_MODEL].
70 //
-----
71 conv1d_forward(act_A, act_B);
72
73 //
-----
74 // Stage 3: Multi-Head Attention + Layer Norm (act_B -> act_A)
75 // Applies RoPE to Q and K, computes scaled dot-product
    attention
76 // across all heads, sums the residual, and applies LayerNorm.
77 //
-----

```



```

78 mha_norm_forward(act_B, act_A);
79
80 //
-----
81 // Stage 4: Feed-Forward Network + Layer Norm (act_A -> act_B)
82 // Two linear layers (d_model -> d_ff -> d_model) with ReLU
83 // activation, residual connection, and LayerNorm.
84 //
-----
85 ffn_norm_forward(act_A, act_B);
86
87 //
-----
88 // Stage 5: Global Average Pooling (act_B -> pooled[D_MODEL])
89 // Averages activations over the SEQ_LEN time dimension,
    reducing
90 // [SEQ_LEN, D_MODEL] to [D_MODEL], matching the software model.
91 //
-----
92 data_t pooled[D_MODEL];
93 pooling(act_B, pooled);
94
95 //
-----
96 // Stage 6: Linear Output Head (pooled -> local_output)
97 // Fully-connected layer: D_MODEL=128 -> OUT_DIM=10.
98 // Weights are statically compiled in from fc_weights.h.
99 //
-----
100 data_t local_output[OUT_DIM];
101 linear2(pooled, local_output, fc_w_flat, fc_b_flat, D_MODEL,
    OUT_DIM);
102
103 //
-----

```

```

104      // AXI Output Write
105      // Burst-write OUT_DIM=10 fixed-point joint angles to DDR.
106      //
      -----
107      TOP_WRITE_OUTPUT:
108      for (int i = 0; i < OUT_DIM; ++i) {
109      #pragma HLS PIPELINE II=1
110          output_axi[i] = local_output[i];
111      }
112  }

```

BIBLIOGRAPHY

- [1] S. A. A. Naqvi et al., “Diabetic foot complications: A major healthcare challenge in pakistan,” *JPM. The Journal of the Pakistan Medical Association*, vol. 71, no. 5, pp. 1478–1483, 2021.
- [2] F. Cordella, E. L’Abbate, et al., “Literature review on needs of upper limb prosthetic users,” *Frontiers in Neuroscience*, vol. 10, p. 209, 2016.
- [3] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, “Efficient transformers: A survey,” *ACM Computing Surveys (CSUR)*, vol. 55, no. 6, pp. 1–28, 2023.
- [4] E. A. Biddiss and T. T. Chau, “Upper limb prosthetics: Critical factors in device abandonment,” *American journal of physical medicine & rehabilitation*, vol. 86, no. 12, pp. 977–987, 2007.
- [5] D. Farina et al., “Surface electromyography: Physiology, engineering, and applications,” *Journal Name*, 2020, PMCID: PMC10807628.
- [6] C. De Luca, *Surface electromyography: Detection and recording*, 2002.
- [7] T. Klotz, L. Gizzi, and O. Röhrle, “Investigating the spatial resolution of emg and mmg based on a systemic multi-scale model,” *Biomechanics and Modeling in Mechanobiology*, vol. 21, pp. 983–997, Apr. 2022.
- [8] R. Merletti and S. Muceli, *Tutorial. surface emg detection in space and time: Best practices tutorial. surface emg detection in space and time: Best practices*.
- [9] M. Reaz, M. Hussain, and F. Mohd-Yasin, “Techniques of emg signal analysis: Detection, processing, classification and applications,” *Biol Proced Online*, vol. 8, pp. 11–35, 2006, Epub 2006 Mar 23. Erratum in: *Biol Proced Online*. 2006;8:163. doi: 10.1251/bpo124. PMID: 16799694; PMCID: PMC1455479.
- [10] J. Smith and A. Taylor, “Transformer-based emg regression for hand motion prediction,” *Taylor & Francis Online*, 2023.
- [11] Open Bionics, *What is a myoelectric hand?* <https://openbionics.com/en/what-is-a-myoelectric-hand/>, Accessed: 2025-12-08, 2025.
- [12] Wikipedia contributors, *I-limb hand — Wikipedia, the free encyclopedia*, https://en.wikipedia.org/wiki/I-Limb_Hand, Accessed: 2025-12-08, 2025.
- [13] “The latest research progress on bionic artificial hands: A systematic review,” *Micromachines*, vol. 15, no. 7, p. 891, 2024, <https://www.mdpi.com/2072-666X/15/7/891>.

- [14] “Design, construction and tests of a low-cost myoelectric thumb,” *Technologies*, vol. 9, no. 3, p. 63, 2021, <https://www.mdpi.com/2227-7080/9/3/63>.
- [15] A. Phinyomark, C. Limsakul, and P. Phukpattaranont, “Feature extraction for myoelectric control systems,” *Journal of Medical and Biological Engineering*, vol. 32, no. 1, pp. 1–13, 2012.
- [16] M. Atzori, A. Gijsberts, et al., “Deep learning for myoelectric-based gesture recognition: A review,” *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 28, no. 4, pp. 755–768, 2020.
- [17] W. Wei, Q. Dai, et al., “Trans-EMG: A Transformer-based model for EMG signal classification,” *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 30, pp. 209–218, 2022.
- [18] Y. Park and S.-M. Lee, “EMG-Transformer: A long-term temporal context-aware model for improved myoelectric-based gesture recognition,” *Sensors*, vol. 22, no. 10, p. 3765, 2022.
- [19] A. Vaswani, N. Shazeer, et al., “Attention is all you need,” in *Advances in neural information processing systems 30 (NIPS 2017)*, 2017, pp. 5998–6008.
- [20] S. Gu, L. Yang, et al., “FPGA-based accelerators for deep learning: A survey,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 31, no. 9, pp. 3438–3456, 2021.
- [21] S. Li, W. Wang, et al., “A survey of fpga-based transformer accelerators,” *Electronics*, vol. 11, no. 12, p. 1891, 2022.
- [22] AMD Xilinx, *Zynq-7000 soc data sheet: Overview*, <https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview>, Accessed: 2025-12-08, 2025.
- [23] N. K. Karnam, S. R. Dubey, A. C. Turlapaty, and B. Gokaraju, “EMGHandNet: A hybrid CNN and bi-LSTM architecture for hand activity classification using surface EMG signals,” *Biocybernetics and Biomedical Engineering*, vol. 42, no. 1, pp. 325–340, Jan. 1, 2022.
- [24] W. Zhang, T. Zhao, J. Zhang, and Y. Wang, “LST-EMG-net: Long short-term transformer feature fusion network for sEMG gesture recognition,” *Frontiers in Neurobotics*, vol. 17, Feb. 28, 2023.
- [25] “Transformer-based hand gesture recognition from instantaneous to fused neural decomposition of high-density EMG signals | scientific reports,” Accessed: Apr. 14, 2026.

- [26] “On-device AI: Quantization-aware training of transformers in time-series,” Accessed: Apr. 14, 2026.
- [27] R. G. Molinari, V. Avilés-Carrillo, G. A. G. D. Villa, and L. A. Elias, “A wearable platform for real-time control of a prosthetic hand by high-density emg,” *medRxiv*, Nov. 2025, Preprint, not peer-reviewed.
- [28] E. Sola-Thomas, M. A. Baset Sarker, and M. Imtiaz, *Fpga-controlled ai vision for prosthetics hand*, IEEE Xplore, Jun. 2023.
- [29] M. Atzori et al., “Electromyography data for non-invasive naturally-controlled robotic hand prostheses,” *Scientific Data*, vol. 1, no. 1, p. 140 053, Dec. 23, 2014.
- [30] C. Lin and Z. He, “A rotary transformer cross-subject model for continuous estimation of finger joints kinematics and a transfer learning approach for new subjects,” *Frontiers in Neuroscience*, vol. 18, p. 1 306 050, Mar. 2024.
- [31] H. Rijkema and M. Girard, “Computer animation of knowledge-based human grasping,” *The International Journal of Robotics Research*, vol. 10, no. 4, pp. 314–326, 1991.
- [32] M. Santello, M. Flanders, and J. F. Soechting, “Postural hand synergies for tool use,” *Journal of Neuroscience*, vol. 18, no. 23, pp. 10 105–10 115, 1998.
- [33] Unity Technologies, *XR Hands (com.unity.xr.hands) documentation (Version X.X)*, <https://docs.unity.com/Packages/com.unity.xr.hands@<version>/manual/index.html>, Unity Manual, 2023.

VITA

Muhammad Quddusi Kashaf

M. Quddusi Kashaf, born in Karachi, is a Computer Engineering senior focused on the intersection of AI and embedded systems. His portfolio spans NLP research (SemEval), medical computer vision (Diabetic Retinopathy), and industrial Non-Destructive Testing. With advanced coursework in Deep Learning, Microcontrollers, and Digital System Design, he brings the interdisciplinary expertise required to bridge complex algorithms with hardware implementation.

Abdullah Amir

Abdullah Amir is a computer engineering student from Karachi who is interested in working at the intersection of AI and hardware, in domains like edge AI, hardware acceleration, and embodied AI. He has previously worked on summer projects developing a Maze Solving robot after his sophomore year, and implementing a Convolutional Neural Network Architecture on FPGA following his junior year, giving him a taste of Embedded Systems and Hardware Acceleration.

Saif Nazir

Saif Nazir is a Computer Engineering Student from Chitral, interested in Robotics, Artificial Intelligence and use of AI in Robotics. He has worked on a summer project in collaboration with ENGRO Polymers where he worked on solution on Non-Destructive Testing and Cleaning of Industrial Storage Tanks. He has also worked on creating a VR based Evacuation Training Simulation for Habib University. He has taken courses in Embedded Systems, Robotics, and Digital Systems Design, which make him capable of taking on this project.

